

Autonome Wegfindung eines Quadrocopters mithilfe des FloodFill Algorithmus

Felix Bischof (Jg. 2005)

Maturitätsarbeit an der Kantonsschule Uster 2023

Betreuende Lehrperson: F. Köhl

Expertin: T. Luternauer

Abgabedatum: 25.10.2023

Abstract

Diese Maturitätsarbeit befasst sich mit der Konstruktion, dem Zusammenbau und der Programmierung eines autonomen Quadrocopters. Dieser soll Hindernissen ausweichen und ein gesetztes Ziel erreichen. Zu Beginn der Arbeit werden die Grundlagen dargelegt, welche für das Verständnis der praktischen Arbeit vorausgesetzt werden. Der Fokus der Programmierung liegt auf der Implementierung eines Wegfindungs Algorithmus und dessen Test im Simulator.

Der Weg vom Start- zum Zielpunkt wird mit dem FloodFill Algorithmus berechnet. Dieser wird mithilfe von DroneKit und ROS in einem Python Skript implementiert, welches die Drohne steuert. Um die Umgebung wahrzunehmen, verwendet die Drohne sechs Abstandssensoren. Die Abstandsdaten dieser Sensoren werden im Python Skript verwendet, um einen Weg zum Ziel zu berechnen. Das Python Skript läuft auf einem in der Drohne verbauten Raspberry Pi.

Während der Arbeit wurden verschiedene Optimierungsmöglichkeiten identifiziert. Zudem kommt die Arbeit zu dem Schluss, dass der implementierte Algorithmus noch Optimierungspotential hat. Zum Beispiel kann der Suchraum des Algorithmus beschränkt werden und der Algorithmus wird somit effizienter.

Der gesamte Source-Code dieses Projekts ist unter <https://github.com/Jack2153/DroneObstacleAvoidance> zu finden.

Inhaltsverzeichnis

Abstract	1
1. Einleitung	4
1.1. Ziele und Fragestellung.....	4
1.2. Aufbau und Methodik.....	5
1.2.1. Schriftliche Arbeit	5
1.2.2. Praktische Arbeit	5
2. Technische Grundlagen	6
2.1. Software.....	6
2.1.1. Gazebo.....	6
2.1.2. Robot Operating System (ROS).....	9
2.1.3. ArduPilot.....	12
2.1.4. Software In The Loop (SITL).....	12
2.1.5. DroneKit.....	13
2.2. Hardware	14
2.2.1. Sensoren.....	14
2.2.2. Flight-Controller (FC)	15
2.2.3. Electronic Speed Controller (ESC)	16
2.2.4. Raspberry Pi.....	16
2.2.5. Global Positioning System (GPS) Modul.....	17
2.2.6. Stromversorgung	17
3. Entwicklung.....	18
3.1. Software.....	18
3.1.1. FloodFill und A* Algorithmen.....	18
3.1.2. Implementierung des FloodFill Algorithmus.....	20

3.1.3.	Hauptprogramm	22
3.1.4.	Testphase im Simulator	25
3.2.	Hardware	26
3.2.1.	Bau des Quadrocopters	26
3.2.2.	Testflüge	29
4.	Ausblick und Optimierungsmöglichkeiten	30
5.	Literaturverzeichnis.....	32
	Abbildungsverzeichnis	39
	Glossar	41
	Eigenständigkeitserklärung	43
	Dank.....	44
	Anhang	45
A.	Komponentenliste	45
B.	Setup Anleitung.....	45
C.	Source-Code	48

1. Einleitung

Quadrocopter haben sich in den letzten Jahren rasant weiterentwickelt und wurden erst im letzten Jahrzehnt benutzerfreundlich und massentauglich. Jedoch sind nicht nur Consumer-Quadrocopter weit verbreitet, sondern auch in der Wissenschaft und Industrie werden Quadrocopter immer öfters als revolutionäres und äusserst hilfreiches Arbeitsinstrument eingesetzt. Von landwirtschaftlichen Anwendungen [1] über die Überwachung von Infrastruktur [2] bis hin zu Unterhaltungs- und Medienanwendungen [3]. Die Fähigkeit autonom zu fliegen, erweitert die Einsatzgebiete auf vielfältige Art und Weise und ermöglicht Anwendungen, welche mit manueller Steuerung nicht umsetzbar oder zu kostspielig sind, wie z.B. die Höhlenrettung, den Transport von Gütern und Personen oder der Überwachung von Gebäuden.

Diese Maturitätsarbeit befasst sich mit der Konstruktion, dem Zusammenbau, der Programmierung und dem Testen eines Quadrocopters, welcher autonom Hindernissen ausweichen kann. Dies markiert den ersten Schritt auf dem weiteren Weg der Forschung bis hin zu einem vollständig autonomen Quadrocopter.

1.1. Ziele und Fragestellung

Das Ziel dieser Maturitätsarbeit besteht darin, einen Quadrocopter zu konstruieren, zu bauen und zu programmieren. Dieser soll autonom zu einem vorgegebenen Ziel fliegen. Dabei muss er selbständig Hindernisse auf dem Weg erkennen, einen alternativen Weg ohne Hindernisse finden und diesem folgen. Daraus ergibt sich die Fragestellung: Wie lässt sich ein autonomer Quadrocopter mit einem geeigneten Wegfindungsalgorithmus konstruieren, zusammenbauen und programmieren?

Zusätzlich soll das Verständnis für das Zusammenspiel von Hardware und Software erweitert werden. Das angeeignete Wissen soll dargelegt und der praktische Teil der Arbeit dokumentiert und erklärt werden.

Das Ziel, den fertigen Quadrocopter über ein gängiges Endgerät wie einen Computer oder ein Smartphone per eigens entwickelte Web-Applikation zu steuern, wurde aufgrund des erhöhten Zeitaufwandes für das Setup des Simulators zurückgestellt. Der Fokus dieser Arbeit liegt somit auf der Erstellung der Flugsoftware und dem Bau des Quadrocopters.

1.2. Aufbau und Methodik

Der Aufbau dieser Maturitätsarbeit ist in einen praktischen und einen schriftlichen Teil unterteilt. Der Grossteil des Arbeitsaufwands entstand im praktischen Teil der Arbeit.

1.2.1. Schriftliche Arbeit

Der schriftliche Teil der Arbeit ist wiederum in zwei Abschnitte unterteilt. Der erste Teil befasst sich mit den Technischen Hintergründen und gibt eine theoretische Grundlage für ein besseres Verständnis der Thematik. Dies beinhaltet auch die Darlegung relevanter Konzepte und Theorien.

Der zweite Teil der schriftlichen Arbeit befasst sich mit dem Entwurf, dem Bau und der Programmierung des Quadrocopters.

1.2.2. Praktische Arbeit

Der praktische Teil der Arbeit ist auch in zwei Abschnitte unterteilt: Die Software und die Hardware. Der Softwareteil besteht darin, verschiedene Algorithmen zu vergleichen, einen geeigneten auszuwählen, diesen zu implementieren und die Implementierung im Simulator zu testen.

Der Hardwareteil setzt sich aus dem Entwurf und der Fertigung verschiedener Teile im 3D-Drucker, dem Bau des Quadrocopters und der Testflüge zusammen.

2. Technische Grundlagen

Im folgenden Abschnitt werden die relevanten Software- und Hardwaretechnischen Grundlagen, die für das Verständnis dieser Arbeit notwendig sind, erläutert.

2.1. Software

Dieser Abschnitt beschreibt folgende Softwarekomponenten und deren Interaktion:

- Gazebo, der Simulator der physikalischen Umgebung und der Sensoren.
- ArduPilot Software In The Loop (SITL), der Simulator der Drohne.
- DroneKit, die Softwarebibliothek für die Steuerung der Drohne.
- Robot Operating System (ROS), die Software für den Informationsaustausch zwischen den Sensoren und dem Skript.

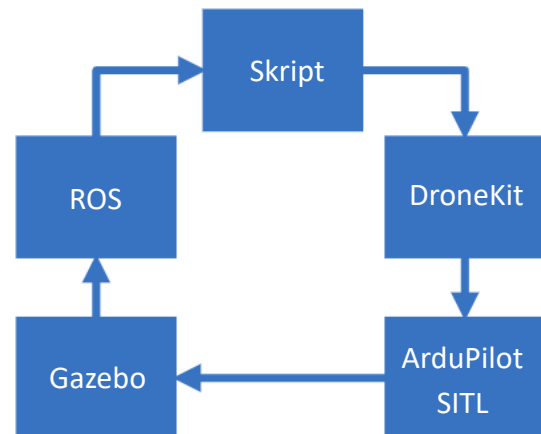


Abbildung 1: Softwarekomponenten und deren Interaktion.

2.1.1. Gazebo

Gazebo ist ein open-source 3D Robotik Simulator, welcher eine integrierte Physik-Engine besitzt. Diese ermöglicht eine realistische Simulation von sich bewegenden Körpern. Gazebo ist darauf ausgelegt unterschiedlichste Roboter zu simulieren, um deren Entwicklung für beliebige Anwendungen zu vereinfachen. Gazebo bietet die Möglichkeit Sensoren zu simulieren, welche auf die virtuelle Welt im Simulator, zum Beispiel ein Hindernis, reagieren. Mithilfe dieser

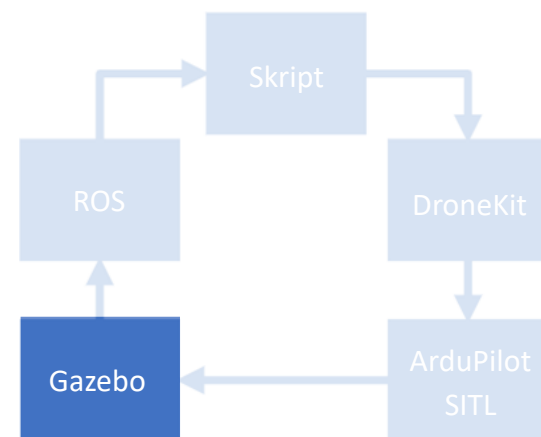


Abbildung 2: Gazebo open-source 3D Robotik Simulator.

Funktionalität wird in Gazebo eine visuelle Drohne simuliert, die mit sechs Abstandssensoren ausgestattet ist. Diese erkennen virtuelle Hindernisse und liefern entsprechende Abstandsdaten. Das Flugverhalten der Drohne wird mit ArduPilot SITL simuliert (Kapitel 2.1.4).

Gazebo visualisiert die Drohne und simuliert die Interaktionen mit den anderen virtuellen Objekten. [4] [5]

Die Hindernisse und das Aussehen der virtuellen Welt werden im Simulation Description Format (SDF) [6] in der Datei `world.sdf` definiert. Diese beinhaltet die Grössen, die Erscheinungsbilder, die Positionen und die physikalischen Eigenschaften der modellierten Objekte. [7] Folgender Beispielcode zeigt die Definition eines nicht durchdringbaren Hindernisses, der Grösse 1m x 6m x 10m:

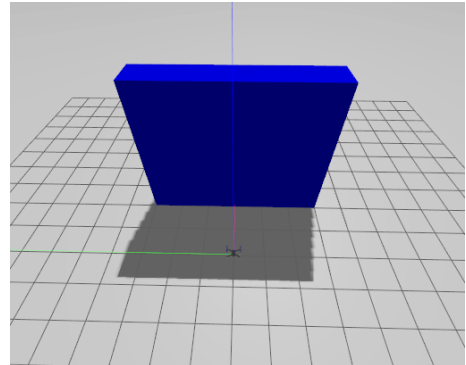


Abbildung 3: Nicht durchdringbares Hindernis der Grösse 1m x 6m x 10m.

```
1 <model name='wall'>
2   <static>true</static>
3   <pose>2.5 0 0 0 0 0</pose><!--pose relative to the world-->
4   <link name='box'>
5     <visual name='visual'>
6       <geometry>
7         <box>
8           <size>1 6 10.0</size>
9         </box>
10      </geometry>
11      <!--add color to our link-->
12      <material>
13        <ambient>0.0 0.0 1.0 1</ambient>
14        <diffuse>0.0 0.0 1.0 1</diffuse>
15        <specular>0.0 0.0 1.0 1</specular>
16      </material>
17    </visual>
18    <collision name='collision'>
19      <geometry>
20        <box>
21          <size>1 6 10.0</size>
22        </box>
23      </geometry>
24    </collision>
25  </link>
26 </model>
```

In der Datei `model.sdf` werden die Attribute des Roboters beschrieben. Anhand dieser Dateien weiss der Simulator wie das Modell des Roboters und der Welt aussehen und miteinander interagieren. [8] Folgender Code zeigt die Definition eines Lidar-Sensors, welcher aber als Abstandssensor benutzt wird, in der Datei `model.sdf`:

```
1 <sensor name='gpu_lidar_front' type='gpu_lidar'>"
2   <pose relative_to='lidar_frame_front'>0 0 0 0 0 0</pose>
3   <topic>sensor_front</topic>
4   <update_rate>10</update_rate>
5   <ray>
6     <scan>
7       <horizontal>
8         <samples>640</samples>
9         <resolution>1</resolution>
10        <min_angle>0</min_angle>
11        <max_angle>0</max_angle>
12      </horizontal>
13      <vertical>
14        <samples>1</samples>
15        <resolution>0.01</resolution>
16        <min_angle>0</min_angle>
17        <max_angle>0</max_angle>
18      </vertical>
19    </scan>
20    <range>
21      <min>0.1</min>
22      <max>100.0</max>
23      <resolution>0.01</resolution>
24    </range>
25  </ray>
26  <always_on>1</always_on>
27  <visualize>true</visualize>
28 </sensor>
```

2.1.2. Robot Operating System (ROS)

Robot Operating System ist die Software (ROS), welche eine Verbindung zwischen verschiedenen Anwendungsprogrammen herstellt. ROS agiert als Vermittler zwischen den verschiedenen Softwarekomponenten und ermöglicht die Kommunikation und den Datenaustausch zwischen den einzelnen Komponenten. Zusätzlich stellt ROS eine Reihe von Diensten und Werkzeugen zur Analyse und Debugging zur Verfügung. [5]

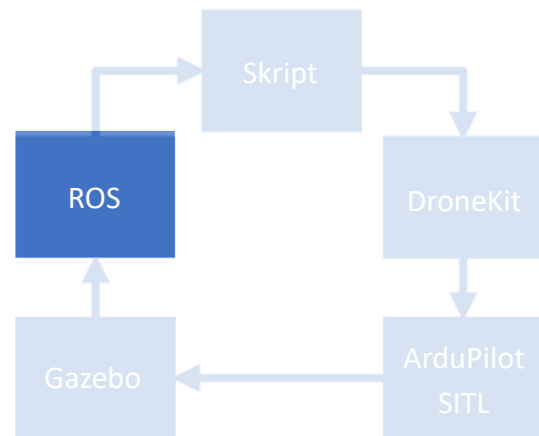


Abbildung 4: ROS stellt eine Verbindung zwischen verschiedenen Anwendungsprogrammen her.

Im Rahmen dieser Arbeit werden die Sensordaten aus dem Gazebo Simulator so weitergeleitet, dass man innerhalb eines Skripts auf diese zugreifen und mit ihnen arbeiten kann.

ROS arbeitet mit Nodes und Topics. Nodes sind Knotenpunkte (Programme) im System. Topics sind Kommunikationskanäle, über welche Nodes miteinander kommunizieren. Das zugrunde liegende Kommunikationsmodell ist Publish-Subscribe. [5] [9]

Sensordaten aus dem Simulator werden auf einem bestimmten Topic veröffentlicht («published»), das heisst, sie werden über diesen Kommunikationskanal gesendet, welcher ein anderer Knotenpunkt abonniert («subscribed») hat und somit die Sensordaten empfängt. Topics und Nodes stellen eine standardisierte und effiziente Kommunikation zwischen den einzelnen Komponenten sicher. [5] [10] [11]

Der Vorteil dieses Systems liegt in der Entkoppelung von Softwarekomponenten, was vor allem in komplexeren Systemen ein Vorteil ist. Die sendende Node, welche Daten veröffentlicht, muss nicht wissen, welche Node diese Daten empfangen soll. So sind Sender- und Empfängernode unabhängig voneinander. [5]

Die auf den Topics veröffentlichten Daten werden unter ROS als strukturierte Daten (Message Types) gesendet. Diese definieren Datentypen und Felder, welche eine Nachricht enthalten kann. In dieser spezifischen Anwendung wird der Nachrichtentyp LaserScan Message benutzt (Abbildung 5), welcher für die Anwendung mit Lidar-Sensoren gedacht ist. Dieser Datentyp hat zehn Datenfelder in welchen verschiedene Informationen übermittelt werden, z.B. die Zeit zwischen zwei Messungen, die Winkelveränderung des Sensors zwischen zwei Messungen und die Distanz einer Messung. [5] [12] [13]

sensor_msgs/LaserScan Message

File: `sensor_msgs/LaserScan.msg`

Raw Message Definition

```
# Single scan from a planar laser range-finder
#
# If you have another ranging device with different behavior (e.g. a sonar
# array), please find or create a different message, since applications
# will make fairly laser-specific assumptions about this data

Header header          # timestamp in the header is the acquisition time of
                        # the first ray in the scan.
#
# in frame frame_id, angles are measured around
# the positive Z axis (counterclockwise, if Z is up)
# with zero angle being forward along the x axis

float32 angle_min       # start angle of the scan [rad]
float32 angle_max       # end angle of the scan [rad]
float32 angle_increment  # angular distance between measurements [rad]

float32 time_increment  # time between measurements [seconds] - if your scanner
                        # is moving, this will be used in interpolating position
                        # of 3d points
float32 scan_time       # time between scans [seconds]

float32 range_min       # minimum range value [m]
float32 range_max       # maximum range value [m]

float32[] ranges         # range data [m] (Note: values < range_min or > range_max should be discarded)
float32[] intensities    # intensity data [device-specific units]. If your
                        # device does not provide intensities, please leave
                        # the array empty.
```

Abbildung 5: LaserScan Message Type [12]

Damit ein ROS Subscriber die Sensordaten aus Gazebo empfangen kann, muss die `ros_gz_bridge` verwendet werden. Diese stellt eine Brücke zwischen Gazebo und ROS dar, sodass diese Nachrichten untereinander austauschen können. [14]

Die Sensordaten jedes einzelnen Sensors werden auf einem eigenen Topic veröffentlicht (Abbildung 6), z. B. `sensor_front` für den Sensor an der Vorderseite der Drohne und `sensor_left` für den Sensor auf der linken Seite.

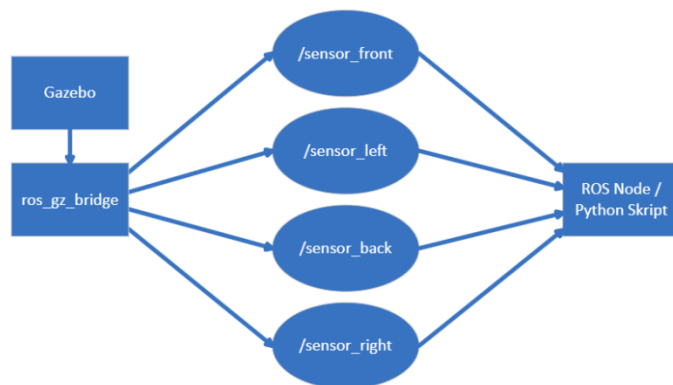


Abbildung 6: Visuelle Darstellung der Kommunikation zwischen Gazebo und ROS

Eine ROS Node immer gleich aufgebaut. Zu Beginn wird der Konstruktor aufgerufen, der die Instanz dieser Klasse initialisiert [9]:

```
1 # ROS Node that implements the Floodfill
2 algorithm and movement of the drone
3 class FloodFillNode(Node):
4
5     # Drone step to count the movements in the movement map
6     robot_step_count = 1
7     def __init__(self):
8         super().__init__('flood_fill_node')
```

Danach kommt der für die jeweilige Klasse spezifische Code. Die Main Funktion initialisiert ROS, erzeugt eine Instanz der Klasse `FloodFillNode`, hält die Node aktiv und beendet diese am Ende des Programms [9]:

```
1 # Main function
2 def main(args=None):
3     rclpy.init(args=args) # Initialize the ROS 2 client library
4     # Create an instance of the FloodFillNode class
5     flood_fill_node = FloodFillNode()
6
7     # Spin the ROS 2 node to keep it alive and responsive to callbacks
8     rclpy.spin(flood_fill_node)
9
10    # Destroy the node to end the program
11    flood_fill_node.destroy_node()
12    rclpy.shutdown() # Shutdown the ROS 2 client library
```

2.1.3. ArduPilot

ArduPilot ist eine open-source Autopilot Plattform für die Entwicklung von autonomen Systemen, wie Flugzeugen, Drohnen oder Booten. Die Plattform stellt die Firmware, die auf der Steuerungshardware läuft, sowie die Softwarewerkzeuge zur Kommunikation mit dem autonomen System zur Verfügung. ArduPilot bietet verschiedene Modi, von vollständig manueller bis vollautomatischer Steuerung. [15]

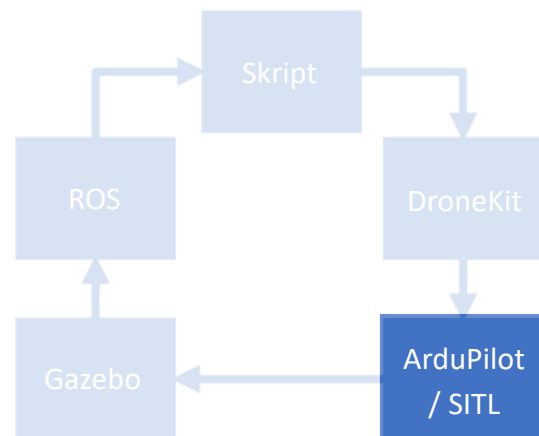


Abbildung 7: ArduPilot stellt die Firmware für den FC bereit.

In dieser Arbeit wird ArduPilot in der Testphase im Simulator mit Software In The Loop (SITL) simuliert um den Quadrocopter ohne Hardware zu testen. Auf dem fertig gebauten Quadrocopter läuft ArduPilot dann auf dem Flight-Controller (FC) (Kapitel 2.2.2), welcher für die Ansteuerung der Motoren zuständig des Quadrocopters zuständig ist. [16]

2.1.4. Software In The Loop (SITL)

Software In The Loop (SITL) ist eine Version von ArduPilot, welche auf dem PC läuft und typischerweise zum Entwickeln und Testen von autonomen Systemen benutzt wird. [16]

In dieser Arbeit wird die Drohne durch SITL simuliert und in Gazebo dargestellt. Über SITL lässt sich die Drohne durch Kommandos direkt steuern. Mithilfe eines Plugins für Gazebo [17] werden die Simulationsdaten aus SITL an den Simulator weitergeleitet und die Bewegungen der Drohne im Simulator dargestellt. [17]

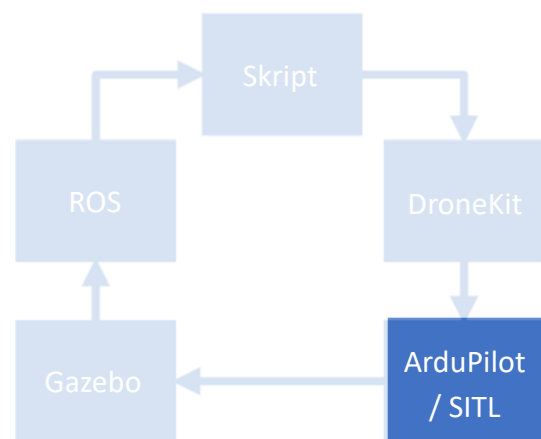


Abbildung 8: SITL ist eine ArduPilot Version, die auf dem PC läuft.

2.1.5. DroneKit

DroneKit ist eine open-source Python Softwarebibliothek, die es Entwicklern ermöglicht spezifische Anwendungen zu erstellen, die mit Drohnen interagieren. Die API dieser Bibliothek ist einfach zu verwenden und ist für den Betrieb mit der ArduPilot-Plattform ausgelegt. Mit DroneKit lassen sich viele Automatisierungen und Funktionen von Drohnen realisieren, von einfachen Aufgaben, wie dem Starten und Landen bis hin zu komplexeren Flugmanövern oder der Durchführung spezieller

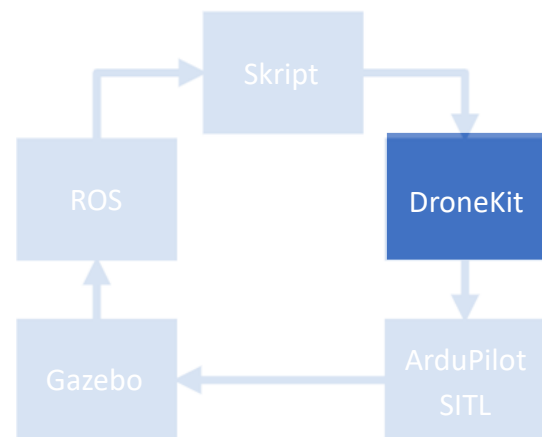


Abbildung 9: DroneKit ist eine Softwarebibliothek zur Kommunikation mit Drohnen

Bewegungen. Zudem lassen sich mithilfe von DroneKit Telemetriedaten abfragen. [18] [19]

In dieser Arbeit wird vor allem die Funktion zur Bewegung und zum Abfragen des Standortes von DroneKit benutzt.

Die einfachste Methode über DroneKit eine Drohne zu bewegen ist die `simple_goto` Funktion. Mit dieser kann man die Drohne zu bestimmten Koordinaten des WGS-84 (ref) Koordinatensystems bewegen. Dieses System ist ein Referenzsystem welches als Grundlage für Positionsangaben auf der Erde und im Erdnahen Raum gilt. Es bildet die Grundlage für die GPS-Koordinaten, welche in Längen- und Breitengrad (Winkelangaben) angegeben werden. [20] [21]

Statt die Drohne mithilfe der `simple_goto` Funktion zu absoluten WGS-84 Koordinaten zu bewegen, kann man sie auch mit der `goto_convenience` Funktion relativ zu ihrer momentanen Position bewegen. Hierbei gibt man an, um wie viel Meter sie nach Norden oder Osten respektiv Süden oder Westen bewegt werden soll. Im Simulator sind Norden und Osten wie in Abbildung 10 ausgerichtet. In der

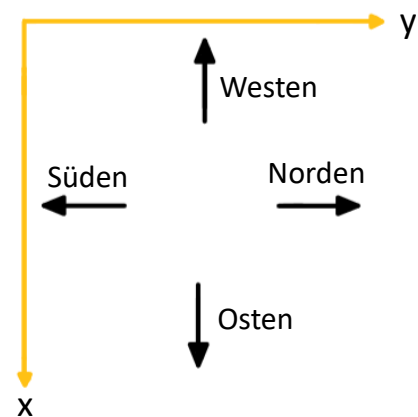


Abbildung 10: Koordinatensystem im Skript und Simulator.

Map des Algorithmus wird eine x und y-Achse verwendet (in Gelb dargestellt). [22]

2.2. Hardware

Dieser Abschnitt erläutert die Hardwarebestandteile aus welchen der Quadrocopter gebaut wird.

2.2.1. Sensoren

Die sechs Abstandssensoren sind die einzige Schnittstelle zwischen der Drohne und ihrer Umgebung. Die richtige und exakte Funktionsweise ist also essenziell für einen erfolgreichen Flug.

Die Sensoren, welche in dieser Arbeit verwendet werden, sind Ultraschall-Abstandssensoren aus einem Arduino Grove-Kit [23]. Der Ultrasonic Distance Sensor V2.0 (Abbildung 11) besteht aus einem Ultraschallsender, Empfänger und einer internen Logik. Bei Aktivierung werden vom Sender Ultraschallwellen ausgesendet. Sobald diese an einem Objekt reflektiert und zurückgeworfen werden, nimmt der Ultraschallempfänger dies wahr. Die interne Logik berechnet mithilfe einer Formel, der verstrichenen Zeit zwischen Ausstrahlung und Empfang und der Schallgeschwindigkeit, wie weit das nächste Objekt entfernt ist. [24] [25]

Diese Abstandsdaten werden zur weiteren Verarbeitung an den Raspberry Pi gesendet (Kapitel 2.2.4).



Abbildung 11: Ultrasonic Distance Sensor V2.0 [24]

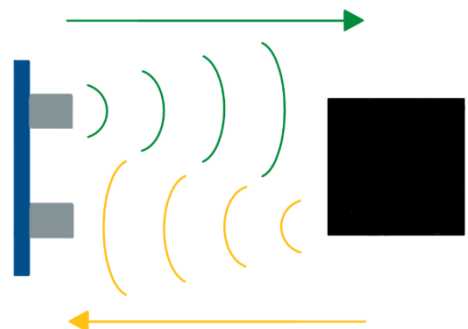


Abbildung 12: Schematische Darstellung der Funktionsweise des Ultraschallsensors. [25]

2.2.2. Flight-Controller (FC)

Der Flight-Controller (FC) ist das Gehirn der Drohne. Dieser ist ein kleiner hochspezialisierter Computer, der für die Steuerung und die Stabilisierung des Fluges verantwortlich ist. Im Normalfall empfängt er Steuerungssignale über einen Empfänger von einem Fernsteuerungssender, welcher ein Pilot steuert. Diese Signale werden auf dem FC verarbeitet und dann an die Motoren weitergeleitet, damit sich die Drohne bewegt. [26]



Abbildung 13: Holybro Kakute F7 Flight-Controller [30]

Statt mit einem manuell gesteuertem Fernsteuerungssender, wird in dieser Arbeit wird mit dem Autopilot System ArduPilot (Kapitel 2.1.3) gearbeitet. Dieses läuft auf dem FC und empfängt die Steuerungssignale, welcher der Raspberry Pi anhand der Sensordaten und des Algorithmus (Kapitel 3.1.2) berechnet hat. [27]

Zusätzlich beinhaltet der FC weitere Sensoren, die zur Flugstabilität beitragen. Für diese Arbeit sind aber nur folgende zwei von Bedeutung:

Das Gyroskop erkennt die Winkeländerungsrate um die drei Achsen x , y und z . Diese Informationen werden genutzt, um die Drohne in der Luft zu stabilisieren und vom Taumeln abzuhalten. Hierzu verarbeitet der der FC die Daten in Echtzeit und wandelt diese in Steuerungsbefehle für die Motoren um, welche die Drehzahlen der Motoren dynamisch anpassen. [26] [28]

Der Beschleunigungssensor bestimmt die lineare Beschleunigung in x -, y - und z -Richtung und zusätzlich den Neigungswinkel im Ruhezustand. Dieser wird dazu benutzt, dass die Drohne nicht gestartet werden kann, wenn sie sich in einem zu steilen Winkel befindet.

Die Kombination aus beiden Sensoren erlaubt es der Drohne die genaue Lage in der Luft zu bestimmen und so konstant mithilfe Steuerungsbefehlen an die Motoren die Lage zu korrigieren. Beide Sensoren sind auf der Inertial Measurement Unit vereint [29]. [26] [28]

Zusätzlich ist auf dem FC ein Barometer verbaut, welcher für die Höhenmessung zuständig ist [30].

Für diese Arbeit wird ein Holybro Kakute F7 Flight-Controller verwendet (Abbildung 13).

2.2.3. Electronic Speed Controller (ESC)

Die Ansteuerung der vier Motoren findet durch Electronic Speed Controllers (ESC) statt. Dies sind Mikrochips, welche die Steuerungsbefehle des FCs (Drehzahländerung) in eine Änderung der Spannung übersetzen und so die Drehzahl der Motoren regulieren. Normalerweise wird die Stromverteilung der Batterie von einem sogenannten Power Distribution Board (PDB) übernommen. Im Rahmen dieser Arbeit wird ein ESC mit Battery Elimination Circuit (BEC) verwendet. Der BEC ist für die Regulierung der Batteriespannung zuständig. Dadurch kann der Flight-Controller von der gleichen Batterie mit einer niedrigeren Spannung versorgt werden. Das verwendete ESC ist ein Hobbywing Xrotor 4in1 ESC. [31] (Abbildung 14)

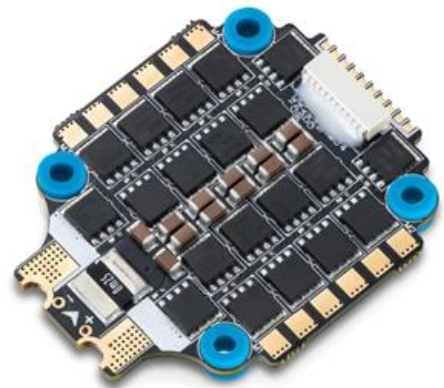


Abbildung 14: Hobbywing Xrotor 4in1 ESC [56]

2.2.4. Raspberry Pi

Raspberry Pi ist ein kompakter Linux basierter Einplatinencomputer. Dieser ist aufgrund der Leistungsfähigkeit in diesem Formfaktor für die Verwendung in eingebetteten Systemen geeignet. [32]

Auf dem Raspberry Pi läuft das selbst entwickelte Python Skript (siehe Kapitel 3), welches anhand der Sensordaten die nächste Bewegung der Drohne berechnet.

Die Sensoren sind direkt an den Raspberry Pi angeschlossen (Kapitel 3.2.1). In dieser Arbeit wird ein Raspberry 3 Model B+ aus dem Jahr 2017 eingesetzt (Abbildung 15).



Abbildung 15: Raspberry Pi 3 Model B+ 2017 [53]

2.2.5. Global Positioning System (GPS) Modul

Das Global Positioning System (GPS) ist ein globales satellitengestütztes System, mit dem man jederzeit die Position in Breitengrad, Längengrad und Höhe bestimmt werden kann. Mit den Entfernungsdaten zu mindestens vier Satelliten und dem Trilaterationsverfahren [33] wird die exakte Position im dreidimensionalen Koordinatensystem berechnet. Die GPS-Daten



Abbildung 16: In der Drohne verbautestes GPS-Modul [54]

werden zur Überprüfung der aktuellen Position der Drohne verwendet. In der Drohne wird ein EM-406A GPS-Modul verwendet (Abbildung 16). [34]

2.2.6. Stromversorgung

Das System wird von einem Lithium-Polymer-Akku (Lipo-Akku) mit Energie versorgt. Der Vorteil dieser Akkus ist die hohe Energiedichte, weshalb auf wenig Gewicht und Volumen viel Energie gespeichert werden kann. Die Kapazität wird in Milliampere-stunden angegeben. Da ein Grossteil der Lipo-Akkus aus mehreren kleinen in Serie geschalteten Akkus bestehen, wird jeweils zusätzlich angegeben, wie viele Zellen in Serie geschaltet werden. Die minimale Grundspannung einer einzelnen Zelle (1s) liegt bei 3.7 V, bei einem 6s (6 Zellen) Akku also bei 22.2 V. Voll aufgeladen erreicht eine einzelne Zelle eine Spannung von 4.2 V, bei einem 6s Akku somit 25.2V. [35]

Die Stromversorgung teilt sich in zwei Teile, einmal wird ein leistungstärkerer 1400 mAh 6s Lipo-Akku für die Versorgung von FC, ESC und den Motoren benutzt und ein schwächerer 2400 mAh 1s Lipo-Akku für die Versorgung des Raspberry Pis.

Der Raspberry Pi benötigt 5 V Eingangsspannung. Um ihn mit einem 1s Lipo zu betreiben, wird ein Purecrea MT3608 Step-Up Spannungsregler [36] (Abbildung 17)



Abbildung 17: Purecrea MT3608 Step-Up Spannungsregler [36]

verwendet, welcher die 4.2V Ausgangsspannung des Akkus auf 5V für den Raspberry Pi umwandelt. [37]

Zusätzlich wird ein TP4056 Charge Controller [38] (Abbildung 18) zwischen dem Akku und dem Raspberry Pi verbaut. Dieser schützt den Akku vor einer Beschädigung durch Tiefenentladung [39].



Abbildung 18: TP4056 Charge Controller [38]

3. Entwicklung

Im folgenden Abschnitt wird die Entwicklung der Drohne aufgeteilt in Software und Hardware beschrieben.

3.1. Software

Die Software ist für die Datenverarbeitung und die Berechnung des Weges zuständig.

3.1.1. FloodFill und A* Algorithmen

Sowohl der FloodFill [40] als auch der A* Algorithmus [41] finden auf einer vorgegebenen Map einen Weg W von einem Startpunkt S zu einem Zielpunkt Z (Abbildung 19). Der A* Algorithmus ist ebenfalls ein weit verbreiteter Wegfindungsalgorithmus [42].

Beim FloodFill Algorithmus wird die Distanz zum Ziel für alle Zellen berechnet. Dazu wird ein Stack eingeführt, auf welchen die noch zu bearbeitenden Zellen liegen. Als erstes wird das Ziel auf den Stack gelegt, da der Algorithmus beim Ziel startet. Der Algorithmus iteriert

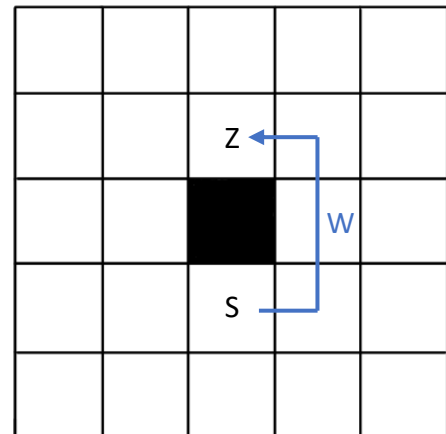


Abbildung 19: Map mit Weg W von S nach Z

über alle Elemente des Stacks und trägt jeweils die Distanz in die Zelle ein. Bei jeder Iteration wird das erste Element gelöscht und es werden weitere Elemente (Nachbarzellen) auf den Stack gelegt. So wird sichergestellt, dass jede Zelle durchgegangen wird. Anschliessend bewegt man sich von S aus entlang der Zellen, die jeweils den kleinsten Abstand zum Ziel haben. [40]

Beim A* Algorithmus wird die Map auch in Zellen unterteilt. Zusätzlich wird ein Wert für jeden Weg von einer Zelle zu Ihrer Nachbarszelle definiert. Dieser bestimmt, wie schwierig es ist, von

einer Zelle zur nächsten zu kommen. So hat der Algorithmus einen ersten Anhaltspunkt, in welche Richtung er gehen soll und in welche nicht. Da die Map gleichmässig ist, sind die Werte für jede Bewegung gleich hoch. Zusätzlich wird eine zweite Grösse eingeführt, die Heuristik. Dies ist eine Schätzung, wie weit eine bestimmte Zelle vom Ziel entfernt ist. Diese zwei Werte addiert ergeben die Kosten für eine Zelle. Nun ist es die Aufgabe vom A* Algorithmus den Weg mit den niedrigsten Kosten zu finden. Mit der Heuristik wird sichergestellt, dass der Algorithmus Zellen priorisiert, die näher am Ziel liegen. [41] [43]

Der A* Algorithmus benutzt zwei Listen, eine offene und eine geschlossene. In der offenen Liste werden die noch nicht besuchten, in der geschlossenen Liste die schon besuchten Zellen verwaltet. Als erstes wird die Startzelle zur offenen Liste hinzugefügt. Solange sich Zellen in der offenen Liste befinden, wird die mit den geringsten Kosten gewählt. Diese wird aus der offenen Liste entfernt und in die Geschlossene eingefügt. Danach werden alle Nachbarzellen betrachtet und für jede überprüft, ob sich diese schon in der geschlossenen Liste befindet. Ist dies der Fall wird diese Zelle ignoriert, da sie schon besucht wurde. Befindet sich eine der Nachbarzellen noch nicht in der offenen Liste, wird diese Zelle der offenen Liste hinzugefügt und die Kosten dieser Zelle berechnet. Zusätzlich wird der Vorgänger dieser Nachbarzelle (aktuelle Zelle) gespeichert, damit am Schluss der kürzeste Weg rekonstruiert werden kann. Sobald das Ziel gefunden wird, rekonstruiert der Algorithmus den Weg. Solange eine Heuristik gewählt wird, welche die tatsächliche Distanz unterschätzt, garantiert A* den kürzesten Weg [44]. [41] [43]

Der Speicherbedarf ist ein wichtiger Faktor für die Beurteilung der Effizienz eines Algorithmus. Der FloodFill Algorithmus speichert die Map mit den Hindernissen und dem Stack, durch welchen sichergestellt wird, dass alle Zellen besucht werden. Im Stack werden maximal $4N^2 - 2(n + m)^2$ (N = Anzahl Zellen, m, n = Breite und Höhe der Map) Elemente gespeichert. Der A* Algorithmus speichert die Map mit den Kosten und zusätzlich zwei Listen. In den beiden Listen zusammen werden maximal N Elemente gespeichert.

¹ Jede Zelle kann maximal vier Mal aufgerufen werden, von jeder Nachbarzelle einmal.

² Die Zellen am Rand der Map haben nur zwei Nachbarzellen und werden somit maximal nur zwei Mal aufgerufen.

Das Laufzeitverhalten ist ein weiterer wichtiger Faktor für die Beurteilung der Effizienz eines Algorithmus. Beim FloodFill Algorithmus wird jede Zelle maximal vier Mal aufgerufen (von jeder Nachbarzelle einmal), das heisst es ist $O(4N) = O(N)$. Beim A* Algorithmus ist das Laufzeitverhalten stark von der verwendeten Heuristik abhängig. Im Worst-Case-Szenario, wenn die Heuristik keine nützlichen Daten liefert, werden alle Zellen betrachtet, d.h. das Laufzeitverhalten ist $O(N)$. In der Realität mit einer entsprechenden Heuristik Funktion ist das Laufzeitverhalten $< O(N)$. [45]

3.1.2. Implementierung des FloodFill Algorithmus

Diese Kapitel beschreibt die Implementierung des FloodFill Algorithmus. Der übrige Teil des Programms wird in Kapitel 3.1.3 beschrieben.

In dieser Arbeit wird der FloodFill Algorithmus implementiert, da dieser für ähnliche Anwendungen weit verbreitet ist und im Vergleich zu ähnlichen Algorithmen für kleine Maps schneller ist [46]. In dieser Arbeit wird der Algorithmus in einer 9x9 Map getestet und angewendet.

Wie zuvor erwähnt, wird in diesem Projekt die Stack basierte Methode des Floodfill Algorithmus gewählt. Die Map wird als zweidimensionales Array implementiert, welches beim Programmstart initialisiert wird.

Abbildung 20 zeigt eine Map mit einem Hindernis, einem Startpunkt S und einem Zielpunkt Z. Die Drohne befindet bei S.

Als erstes werden innerhalb der `floodfill` Funktion zwei Variablen mit Höhe und Breite der Map definiert. Diese dienen später zur Überprüfung, ob sich eine gegebene Zelle in der Map befindet oder nicht. Anschliessend werden durch die Arrays `dx` und `dy` die Richtungen definiert, in welche sich der Algorithmus durch die Map bewegen kann. Danach wird der im Kapitel 3.1.1 erwähnte Stack initialisiert.

Dabei wird, wie in Abbildung 21 dargestellt, das Ziel mit der Distanz 0 auf den Stack gelegt.

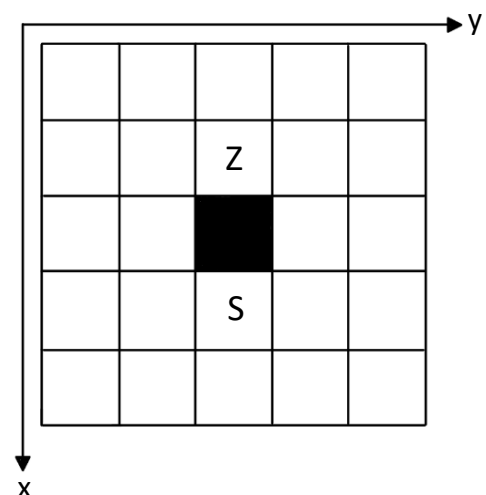


Abbildung 20: Map mit Hindernis, Startpunkt S und Zielpunkt Z.

Anschliessend wird über alle Elemente des Stacks iteriert. Bei jeder Iteration wird das oberste Element vom Stack genommen und die Distanz zum Ziel aktualisiert. Dann werden für alle vier Nachbarzellen (Abbildung 22), die sich in der Map befinden und kein Hindernis sind, jeweils ein Element auf den Stack gelegt und die Distanz um eins erhöht (Abbildung 23).

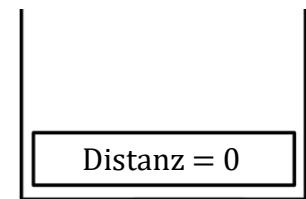


Abbildung 21: Stack zu Beginn des Algorithmus.

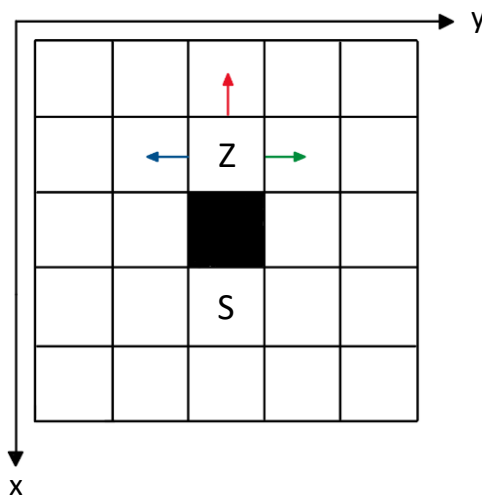


Abbildung 22: Richtungen der drei möglichen Nachbarzellen

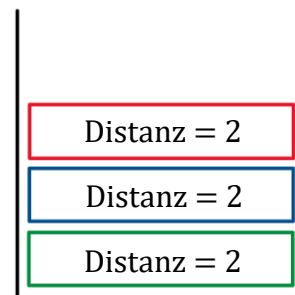


Abbildung 23: Drei elemente mit der Distanz liegen auf dem Stack

Die Koordinaten der vier Nachbarzellen werden mit den Richtungsarrays dx und dy berechnet. Die Koordinaten new_x und new_y der ersten Nachbarzelle berechnen sich wie folgt: Zur aktuellen x- und y-Koordinate wird jeweils die erste Position der Richtungsarrays dx und dy addiert. Für die zweite Nachbarzelle, wird jeweils die zweite Position addiert. Analog geschieht dies für die dritte und vierte Nachbarzelle. Die Richtungsarrays dx und dy sehen wie folgt aus:

```
1 # Define arrays to specify 4-way (non-diagonal) movement directions
2 dx = [0, 0, 1, -1] # Movement directions of the x-axis
3 dy = [1, -1, 0, 0] # Movement directions of the y-axis
```

Abbildung 24 zeigt die aus dx und dy resultierenden Bewegungsrichtungen.

3.1.3. Hauptprogramm

Um das Verhalten der Drohne zu kontrollieren, werden zusätzliche Funktionen definiert. Mithilfe der `arm_and_takeoff` Funktion [47] lässt sich die Drohne starten und steigt auf eine vorgegebene Höhe. Zusätzlich wird die in Kapitel 2.1.5 erwähnte `goto_convenience` Funktion [22] definiert.

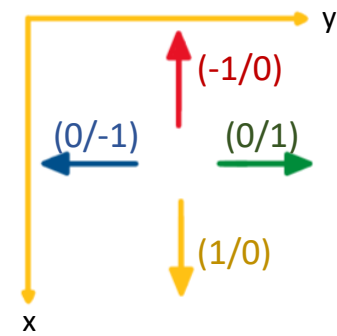


Abbildung 24: Aus dx und dy resultierende Richtungen.

Damit die Hindernisse in der Map eingetragen werden können, muss innerhalb des Skripts auf die Abstandsdaten des Simulators zugegriffen werden. Dies geschieht wie im Kapitel 2.1.2 erwähnt über ROS. Die Erzeugung der Subscriber benötigt den Nachrichtentyp, das Topic, die Callback Funktion und die Länge der Warteschlange. Der Nachrichtentyp `LaserScan` muss angegeben werden, um sicherzustellen, dass der Datenaustausch funktioniert. Das Topic muss angegeben werden, damit der Subscriber weiss, wo die auszulesenden Daten veröffentlicht werden. Im folgenden Code werden die Nachrichten auf dem Topic `/sensor_right` veröffentlicht. Die Funktion `sensor_right_callback` wird bei jedem Eintreffen einer neuen Nachricht auf dem abonnierten Topic aufgerufen. Diese Funktion ist für die Datenverarbeitung zuständig. Falls mehr Nachrichten veröffentlicht als verarbeitet werden können, werden diese in der Warteschlange zwischengespeichert. So wird sichergestellt, dass keine Nachrichten verloren gehen. Ist die Warteschlange voll, wird jeweils die älteste Nachricht gelöscht und die Neuste hinzugefügt.

```
1 # Create a subscriber that listens to the /sensor_right topic of ROS2
2 # where the right sensor publishes the data
3 self.subscription_right = self.create_subscription(
4     LaserScan,
5     '/sensor_right',
6     self.sensor_right_callback,
7     10)
```

Nach der Instanziierung der Subscriber werden die verschiedenen Maps initialisiert und die Koordinaten der Drohne in die Mitte der Map gesetzt. Nachdem überprüft wird, ob die Drohne

schon in der Luft ist, werden die Koordinaten des Ziels durch einen Benutzer eingegeben. Danach werden alle Callback-Funktionen, welche zu den Subscribern gehören, definiert. In diesen wird die Distanz zu einem Objekt in einer Variablen gespeichert. Danach wird überprüft, ob die Distanz unendlich gross ist. Ist dies nicht der Fall, wird die Funktion zum Hinzufügen eines Hindernisses mit Übergabe des Sensornamens und der Distanz aufgerufen.

Die Funktion `add_obstacle` fügt ein Hindernis in der Map ein. Nach der Überprüfung, ob das Hindernis in der Map liegt, wird die Zelle, in der sich das Hindernis befindet, mit -1 markiert.

Es werden drei Maps initialisiert. In der `map_distance` werden die Distanzen und Hindernisse nach jedem Durchgang des FloodFills gespeichert, in der `map_movement` werden die Bewegungen der Drohne dargestellt und in der `map_obstacles` werden die Orte der Hindernisse gespeichert. Zusätzlich wird in jeder Durchführung des FloodFills eine `temp_map_distance` initialisiert. Nachdem diese initialisiert wurde, werden die Hindernisse der `map_obstacles` in die `temp_map_distance` kopiert. Danach wird der FloodFill Algorithmus für diese Map durchgeführt. Das Resultat davon ist eine Map, in welcher die Hindernisse und die Abstände von jeder Zelle zum Ziel eingetragen sind (Abbildung 25). Im nächsten Schritt wird die `map_distance` mit den Daten der `temp_map_distance` aktualisiert und enthält somit nun die aktuellen Distanzen und Hindernisse. Zusätzlich wird in der `map_distance` das Ziel mit einer Null markiert.

3	2	1	2	3
2	1	Z	1	2
3	2		2	3
4	3	S	3	4
5	4	5	4	5

Abbildung 25: Map mit eingetragenen Distanzen zum Ziel für jede Zelle.

Sobald der FloodFill Algorithmus in der `distance_map` durchgeführt wurde und alle Maps aktualisiert wurden, wird überprüft, ob sich die Drohne bereits am Ziel befindet, ist dies nicht der Fall, wird die Drohne durch die `move_robot` Funktion bewegt. Die Drohne kann sich nur in eine der vier benachbarten Zellen bewegen und bewegt sich in die Zelle mit dem kleinsten Abstand zum Ziel. Die Vorgehensweise, die Koordinaten dieser vier benachbarten Zellen zu

berechnen, ist gleich wie beim FloodFill. Es werden wieder zwei Bewegungsarrays dx und dy definiert. Im Unterschied zur `floodfill` Funktion wird aber zu Beginn der `move_robot` Funktion eine Variable definiert, welche die minimale Distanz speichert und ein Array, welches die Positionen mit minimaler Distanz speichert, falls es mehrere Zellen mit gleichem Abstand zum Ziel gibt. Zu Beginn wird die minimale Distanz auf unendlich gesetzt. Und das Array mit den Positionen leer gelassen. Jedes Mal, wenn sich die Drohne bewegt, somit die `move_robot` Funktion ausgeführt wird, werden diese Variable und das Array initialisiert. Danach wird überprüft, ob die benachbarte Zelle eine kleinere Distanz zum Ziel beinhaltet als die Anfangs definierte minimale Distanz. Für die erste benachbarte Zelle ist dies immer der Fall, da die minimale Distanz am Anfang auf unendlich gesetzt wird und keine Zelle einen unendlichen Abstand zum Ziel hat. Somit wird die minimale Distanz aktualisiert auf die Distanz der ersten benachbarten Zelle gesetzt. Zusätzlich wird diese Zelle in das Array gesetzt mit den minimalen Distanzen gesetzt. Für die zweite benachbarte Zelle ist die minimale Distanz die der ersten berechneten benachbarten Zelle. Somit wird die zweite benachbarte Zelle nun mit dieser verglichen. Ist der Abstand in dieser kleiner wird die minimale Distanz aktualisiert und nur diese zweite berechnete Zelle wird in das Array mit Zellen mit minimaler Distanz gesetzt. So werden alle benachbarten Zellen miteinander verglichen und die Zelle mit der kleinsten Distanz wird ermittelt. Zusätzlich wird überprüft, ob die nächste benachbarte Zelle den gleichen minimalen Abstand hat wie die zuvor überprüfte Zelle. Ist dies der Fall, so wird diese Zelle auch in das Array der Zellen mit minimalem Abstand gesetzt. Danach wird dann überprüft, ob dieses Array einen Inhalt hat, ist dies der Fall wird zufällig entschieden (Abbildung 26), in welche Zelle die Drohne sich bewegen soll. Dies ist möglich, da sich in diesem Array nur Zellen mit gleichem Abstand zum Ziel befinden, es spielt also keine Rolle, an welche Position die Drohne sich bewegt.

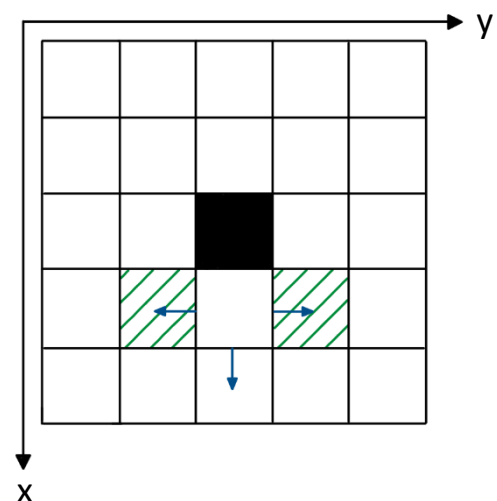


Abbildung 26: Mögliche Richtungen in welche sich die Drohne bewegen kann.

Die Koordinaten der neuen Zelle, in welche sich die Drohne bewegen soll, sind klar. Um dies nun in eine

Bewegung der Drohne umzuwandeln muss ermittelt werden in welcher Richtung sich die neue Zelle befindet. Dies ist der Fall, da sich die Drohne, mit der in Kapitle 2.1.5 erklärten `goto_convenience` Funktion, nur um eine gewisse Anzahl Meter nach Norden und Osten relativ zu ihrer aktuellen Position bewegen lässt. Um herauszufinden in welcher Richtung sich die neue Zelle, relativ zur momentanen Position der Drohne, befindet, wird der Unterschied der momentanen x- und y-Koordinate und der neuen x- und y-Koordinate berechnet. Diese Differenzen werden in zwei Variablen `movement_x` und `movement_y` gespeichert. In einer Reihe von Überprüfungen wird überprüft, welche Werte diese Variablen angenommen haben, und anschliessend die Drohne in die entsprechende Richtung bewegt. Wie im Kapitel 2.1.5 beschrieben, unterscheiden sich die Koordinatensysteme des Arrays und des Simulators. Bewegt sich die Drohne in der Map auf der x-Achse ($\text{movement_x} \neq 0$) muss sich die Drohne im Simulator nach Osten oder Westen bewegen. Bewegt sie sich auf der y-Achse ($\text{movement_y} \neq 0$) muss sich die Drohne im Simulator nach Norden oder Süden bewegen.

3.1.4. Testphase im Simulator

Um nicht mit einer realen Drohne testen zu müssen und Abstürze und Materialverschleiss zu vermeiden, wird der implementierte Algorithmus im Simulator Gazebo getestet. Das Python Skript wurde in Form einer ROS Node geschrieben, um wie schon im Kapitel 2.1.2 erwähnt, die Kompatibilität mit dem Simulator zu gewährleisten.

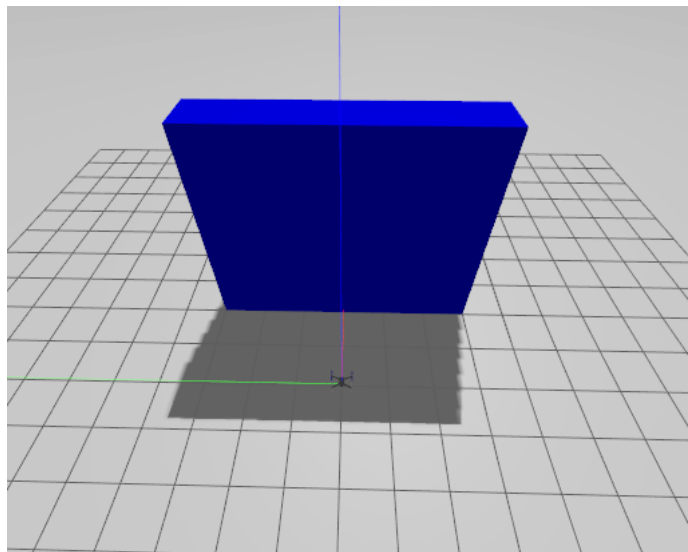


Abbildung 27: Wand als Hindernis im Simulator.

Um schnell Fortschritte zu erzielen, wird die simulierte Drohne anfangs nur mit einem Sensor ausgestattet und das Hindernis bestand aus einer einfachen Wand (Abbildung 27).

Im Simulator konnte das gesamte Verhalten der Drohne simuliert werden. So konnte jegliches Fehlverhalten der Drohne, welches aufgrund des Python Skripts entstand, durch Verbesserungen und Optimierungen beseitigt werden.

Die Nodes, die miteinander kommunizieren sind in der Gazebo Simulator (Publisher) und Python Skript (Subscriber). Die genaue funktionsweise dieser Node/Skripts wird in Kapitel 3.1 genauer erläutert.

3.2. Hardware

3.2.1. Bau des Quadrocopters

Für den Bau des Quadrocopters wurde eine Liste aller benötigten Komponenten (Anhang A) erstellt und auf Kompatibilität überprüft. Anhand dieser Liste wurden Komponenten bei verschiedenen Anbietern eingekauft und bereits Vorhandene aufbereitet. (Anhang A)

Das Frame des Quadrocopters wurde mit TinkerCad [48] konstruiert und mit einem 3D-Drucker [49] unter Verwendung von Polyactid Acid (PLA) [50] ausgedruckt. PLA ist ein synthetisches Polymer aus welchem Bio-Kunststoff [51] hergestellt wird, dieser ist einfach zu drucken, führt zu robusten Bauteilen und lässt sich gut verleimen.

Als erstes wird der 3D gedruckte Rahmen zusammengebaut. Dafür werden die Arme auf die untere Basisplatte gelegt, ausgerichtet und mithilfe von Leim und senkrechten Stiften befestigt (Abbildung 28).



Abbildung 28: Arme an der Basisplatte befestigt

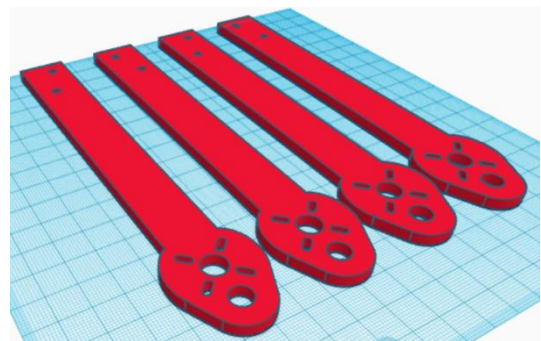


Abbildung 29: Konstruktion der Arme in TinkerCad [48]

Danach werden die ebenfalls 3D gedruckten Halterungen für die Sensoren zwischen den Armen verklebt. Anschliessend wird von oben die zweite Basisplatte draufgelegt und von unten verschraubt. Für zusätzliche Stabilität gehen die Schrauben auch durch die Halterungen für die Sensoren. Im nächsten Schritt wird mithilfe von je vier Schrauben die Motoren auf den Armen befestigt und an das ESC angelötet, welches in der Mitte der Basisplatte von unten angeschraubt wird. Über diese wird der FC angebracht (Abbildung 30).

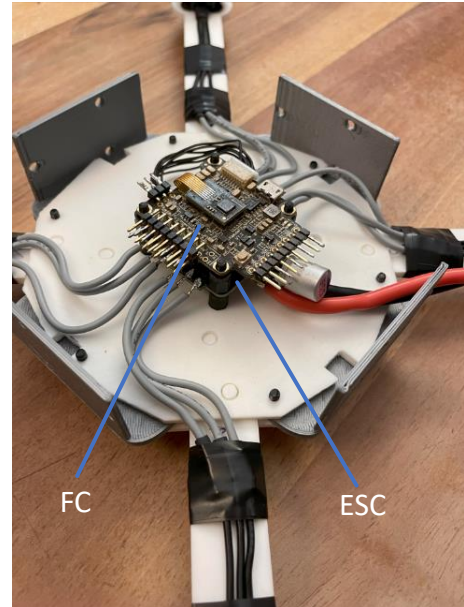


Abbildung 30: Motoren an das ESC angeschlossen, FC darüber angebracht.

Danach werden die Sensoren an den zuvor befestigten Haltern befestigt. Darauffolgend werden, wie in Abbildung 31 schematisch dargestellt, alle Kabel für die Sensoren und die General Purpose Input Output (GPIO) Pins des Raspberry Pis angelötet. Jeder Sensor benötigt jeweils ein Spannungskabel (VCC, rot) ein Groundkabel (GND, schwarz) und ein Signalkabel (SIG, gelb), über welches die Distanz kommuniziert wird.

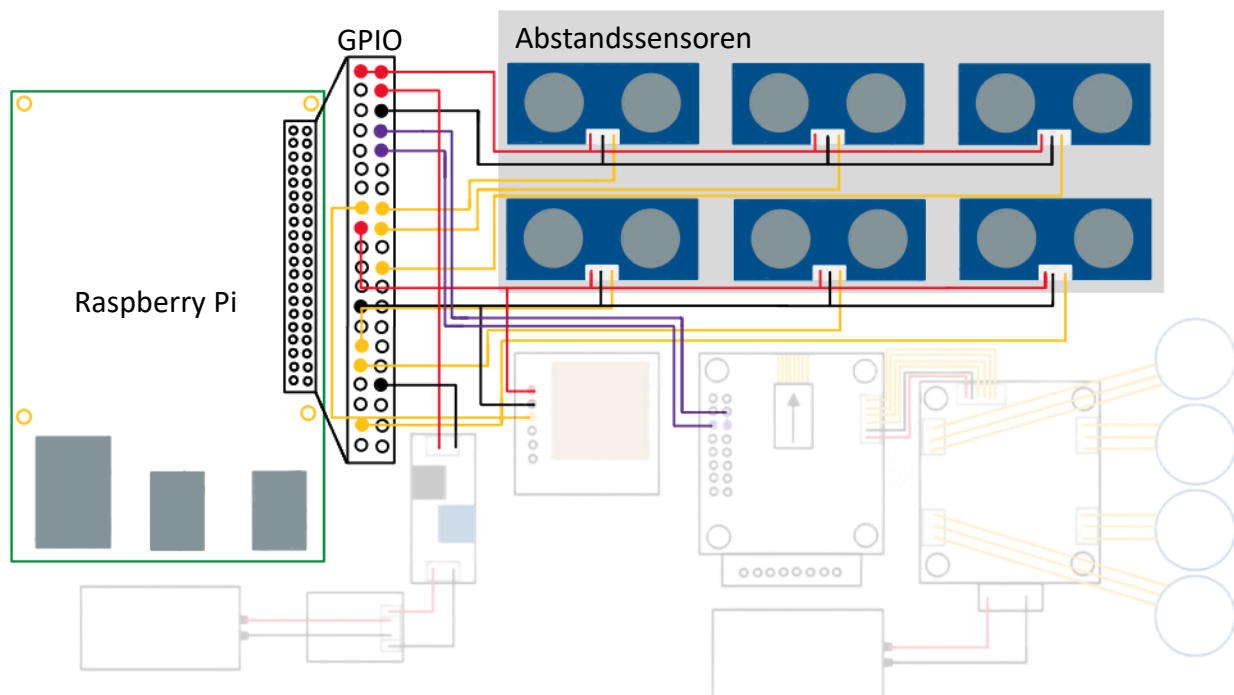


Abbildung 31: Gelb: Signal; Rot: Spannung; Schwarz: Ground; Volett: Receiver (RX) und Transmitter (TX) [55]

Nach dem Verlegen dieser Kabel werden die zwei Boards für die Regulierung des Akkus für den Raspberry Pi verlötet und angeschlossen. Dann wird der FC an den Raspberry Pi angeschlossen, damit dieser die Steuerungsbefehle des Raspberry Pi bekommt. Hierfür wird der Transmitter (TX) Pin des FCs und den Receiver (RX) Pin des Raspberry Pi gelötet und andersherum (Abbildung 33). Zusätzlich wird das zuvor beschriebene GPS-Modul angeschlossen und verbaut. Dieses wird, wie in Abbildung 34 dargestellt, mit einem Spannungskabel (rot), einem Groundkabel (schwarz) und einem Kabel zur Kommunikation der Daten angeschlossen. Im letzten Schritt wird eine Halterung für die beiden Lipo-Akkus gebaut, damit diese während dem Flug sicher verstaut sind.

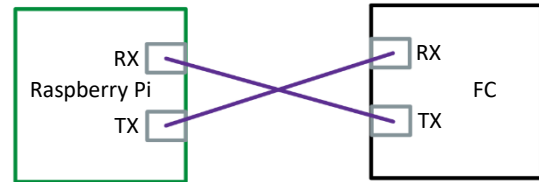


Abbildung 33: Gekreuzte RX TX Verbindung

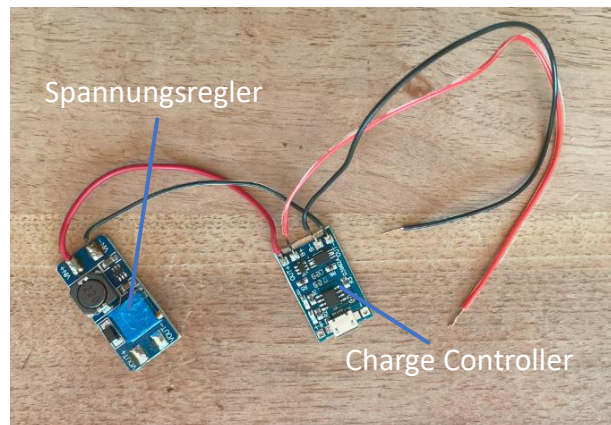


Abbildung 32: Step-Up Spannungsregler und Charge Controller

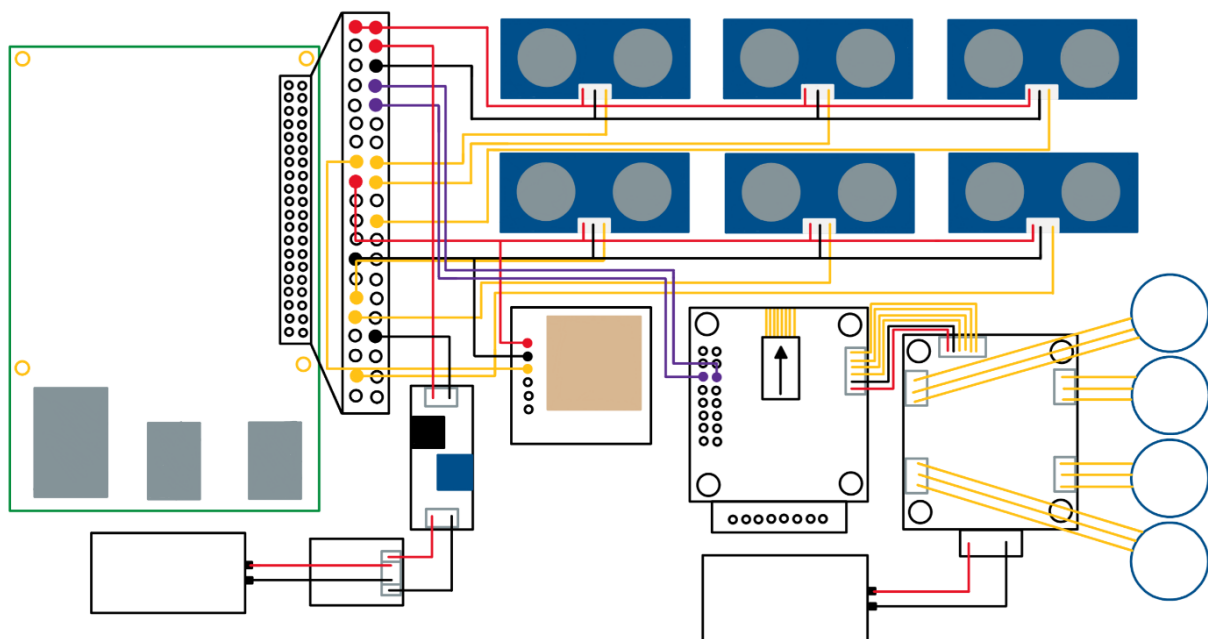


Abbildung 34: Überblick über den Aufbau der Elektronik



Abbildung 35: Vollständig zusammengebaute Drohne

3.2.2. Testflüge

Der implementierte Algorithmus wurde durch Testflüge im Simulator getestet. Durch genaue Beobachtungen und Telemetriedaten der simulierten Drohne konnten Fehler im Python Skript, welches die Drohne steuert, verbessert werden.

Sobald sich die Drohne im Simulator in der Luft befindet, ist zu beobachten, dass sie leicht vor und zurück bewegt. Dies kann zu unkontrollierten Positionsänderungen führen, welche nur mit grossem Aufwand korrigiert werden können.

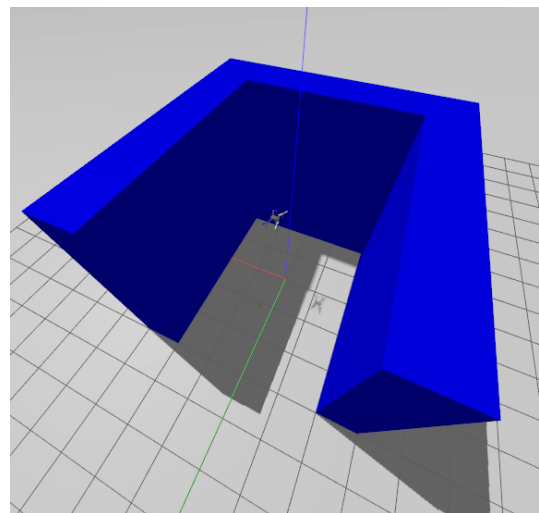


Abbildung 36: Testflug im Simulator mit U-förmigem Hindernis

Die Grundfunktionen des gebauten Quadrocopters wurden durch manuell durchgeführte Testflüge erfolgreich überprüft. Jedoch war ein autonomer Testflug nicht möglich. Der Grund dafür ist die nur einseitig funktionierende Kommunikation zwischen Raspberry Pi und FC.

Zusätzlich hat das Auslesen der GPS-Daten nicht funktioniert. Die Ursache dieser Probleme konnten in der zur Verfügung stehenden Zeit nicht gefunden werden.

Das verwendete GPS-Modul hat eine Genauigkeit von 5 – 10 Metern [52]. Folglich müsste die Grösse einer Zelle der Map erhöht werden im Vergleich zu den im Source-Code definierten 1 Meter Zellen.

Im Verlauf der Arbeit wurde der Raspberry Pi aufgrund von Startproblemen durch einen gleicher Art ausgetauscht. Vermutlich kann die Ursache auf eine fehlerhafte Verbindung zurückgeführt werden. Die aktuellen Kommunikationsprobleme des Raspberry Pis scheinen einen anderen Grund zu haben.

4. Ausblick und Optimierungsmöglichkeiten

Während der Umsetzung dieser Maturitätsarbeit wurden verschiedene Optimierungsmöglichkeiten identifiziert, welche im folgenden Kapitel kurz erläutert werden.

In der aktuellen Version wird der Algorithmus auf die ganze Karte angewendet, was nicht unbedingt nötig ist. Die Abstandswerte, welche sich nicht zwischen der Drohne und dem Hindernis befinden, sind für den jeweiligen Schritt irrelevant. Der Algorithmus wäre effizienter und schneller, würden diese Abstände nicht berechnet werden. Alternativ könnte man den in Kapitel 3.1.1 erklärten A* Algorithmus implementieren, da dieser nur einen Weg von der Drohne bis zum Ziel berechnet und nicht alle möglichen Wege, wie dies beim FloodFill Algorithmus der Fall ist. So könnte man sich Ressourcen und Zeit in der Durchführung des Algorithmus sparen.

Eine weitere naheliegende Optimierung ist die Erweiterung des Algorithmus in die dritte Dimension. Eine Drohne ist ein Fluggerät und kann sich in drei Dimensionen bewegen. Aufgrund der begrenzten zur Verfügung stehenden Zeit wurde der Algorithmus auf zwei Dimensionen beschränkt. Der Code ist flexibel aufgebaut und kann in die dritte Dimension erweitert werden.

Ein Webinterface oder eine Applikation zur Zielsetzung und Darstellung der Karte und des Flugweges würde die Benutzerfreundlichkeit erhöhen. Über eine Bluetooth-Verbindung oder

ein lokales Netzwerk könnten Informationen zwischen der Drohne und dem Endgerät ausgetauscht werden.

Einer der begrenzenden Faktoren im gebauten System sind die Sensoren. Diese sind statisch und messen die Distanz innerhalb eines begrenzten Sichtfeldes. Man könnte die Sensoren durch Servos konstant um einen bestimmten Winkel hin und her schwenken. So vergrössert sich das Bild. Wenn sich jeder der vier Sensoren jeweils um 90 Grad dreht, hätte die Drohne ein 360 Grad Bild, was es ermöglicht, mehrere oder grössere Hindernisse in einem Schritt zu erkennen. So müsste die Drohne das Gelände weniger «abtasten» und wäre schneller am Ziel. Um die eben erwähnte Funktionsweise zu erreichen, müsste ein zweiter Algorithmus zur Verarbeitung der Daten implementiert werden. Dieser würde, mit dem zur jeweiligen Messung gehörenden Winkel und dem Abstand, die Hindernisse auf der Karte eintragen.

Um den ganzen Raum auf einmal zu erfassen, wäre es ebenfalls möglich einen 3D-LIDAR-Sensor zu verbauen. Dieser misst mithilfe von Laserstrahlen und der gemessenen Zeit zwischen Ausstrahlung und Reflektion die Umgebung aus. Der Vorteil wäre, dass dieser den gesamten Raum schneller ausmessen kann. Die Herausforderung hier wäre der Algorithmus zur Verarbeitung der Daten.

5. Literaturverzeichnis

- [1] Bundesinformationszentrum Landwirtschaft, „Wofür braucht man Drohnen in der Landwirtschaft?“, 2023. [Online]. Available: <https://www.landwirtschaft.de/landwirtschaft-verstehen/wie-funktioniert-landwirtschaft-heute/wofuer-braucht-man-drohnen-in-der-landwirtschaft>. [Zugriff am 22 10 2023].
- [2] Fraunhofer IMS, „Infrastrukturüberwachung“, 2023. [Online]. Available: <https://www.ims.fraunhofer.de/de/Geschaeftsfeld/Industry/Nachhaltige-Produktion/Infrastrukturueberwachung.html>. [Zugriff am 22 10 2023].
- [3] SWISS DRONE SHOW AG, „SWISS DRONE SHOW“, 2023. [Online]. Available: <https://www.swissdroneshow.com/>. [Zugriff am 22 10 2023].
- [4] Open Robotics, „Gazebo Simulator Home“, [Online]. Available: <https://gazebo.org/home>. [Zugriff am 19 10 2023].
- [5] L. Poubel, „Open Source Robotics: Getting Started with Gazebo and ROS 2“, InfoQ, 7 5 2019. [Online]. Available: <https://www.infoq.com/articles/ros-2-gazebo-tutorial/>. [Zugriff am 20 10 2023].
- [6] Open Source Robotics Foundation, „SDFormat“, 2020. [Online]. Available: <http://sdformat.org/>. [Zugriff am 19 10 2023].
- [7] Open Robotics, „SDF worlds“, [Online]. Available: https://gazebo.org/docs/garden/sdf_worlds. [Zugriff am 20 10 2023].
- [8] Open Robotics, „Building Your Own robot“, [Online]. Available: https://gazebo.org/docs/garden/building_robot. [Zugriff am 20 10 2023].

- [9] Open Robotics, „Understanding nodes,“ 2023. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Nodes/Understanding-ROS2-Nodes.html>. [Zugriff am 20 10 2023].
- [10] Open robotics, „Understanding topics,“ 2023. [Online]. Available: <https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html>. [Zugriff am 20 10 2023].
- [11] GeekforGeeks, „ROS Publishers using Python,“ 28 2 2022. [Online]. Available: <https://www.geeksforgeeks.org/ros-publishers-using-python/>. [Zugriff am 20 10 2023].
- [12] Open Robotics, „sensor_msgs/LaserScan Message,“ 28 2 2022. [Online]. Available: https://docs.ros.org/en/melodic/api/sensor_msgs/html/msg/LaserScan.html. [Zugriff am 20 10 2023].
- [13] D. Kurzaj, „Messages,“ Open Robotics, 26 8 2016. [Online]. Available: <https://wiki.ros.org/Messages>. [Zugriff am 20 10 2023].
- [14] Open Robotics, „Bridge communication between ROS and Gazebo,“ GitHub, 15 6 2023. [Online]. Available: https://github.com/gazebo-sim/ros_gz/blob/ros2/ros_gz_bridge/README.md. [Zugriff am 22 10 2023].
- [15] ArduPilot, „ArduPilot,“ 2023. [Online]. Available: <https://ardupilot.org/>. [Zugriff am 20 10 2023].
- [16] ArduPilot Dev Team, „SITL Simulator (Software in the Loop),“ ArduPilot, 2023. [Online]. Available: <https://ardupilot.org/dev/docs/sitl-simulator-software-in-the-loop.html>. [Zugriff am 20 10 2023].
- [17] ArduPilot Dev Team, „Using SITL with Gazebo,“ ArduPilot, 2023. [Online]. Available: <https://ardupilot.org/dev/docs/sitl-with-gazebo.html>. [Zugriff am 20 10 2023].

- [18] 3D Robotics, „About DroneKit,“ 2016. [Online]. Available: <https://dronekit-python.readthedocs.io/en/latest/about/overview.html>. [Zugriff am 22 10 2023].
- [19] 3D Robotics, „DroneKit-Python’s documentation,“ 2016. [Online]. Available: <https://dronekit-python.readthedocs.io/en/latest/>. [Zugriff am 18 10 2023].
- [20] Rainald62, „World Geodetic System 1984,“ Wikipedia, 19 10 2023. [Online]. Available: https://de.wikipedia.org/wiki/World_Geodetic_System_1984. [Zugriff am 22 10 2023].
- [21] 3D Robotics, „Guiding and Controlling Copter,“ 2016. [Online]. Available: https://dronekit-python.readthedocs.io/en/latest/guide/copter/guided_mode.html#guided-mode-copter-position-control. [Zugriff am 22 10 2023].
- [22] 3D Robotics, „Guided Mode Movement and Commands (Copter),“ 2016. [Online]. Available: <https://dronekit-python.readthedocs.io/en/latest/examples/guided-set-speed-yaw-demo.html>. [Zugriff am 19 10 2023].
- [23] hushuxu, „Grove Beginner Kit For Arduino,“ Seeed Studio, 14 2 2023. [Online]. Available: <https://wiki.seeedstudio.com/Grove-Beginner-Kit-For-Arduino/>. [Zugriff am 22 10 2023].
- [24] gunengyu, „Grove - Ultrasonic Ranger,“ Seeed Studio, 1 3 2023. [Online]. Available: https://wiki.seeedstudio.com/Grove-Ultrasonic_Ranger/. [Zugriff am 22 10 2023].
- [25] Shawn, „Types of Distance Sensors and How to Select One?,“ Maazine 7, 23 12 2019. [Online]. Available: <https://www.seeedstudio.com/blog/2019/12/23/distance-sensors-types-and-selection-guide/>. [Zugriff am 22 10 2023].
- [26] OscarLiang.com, „Flight Controller Explained: Understanding FPV Drone Control Systems,“ 13 9 2023. [Online]. Available: <https://oscarliang.com/flight-controller-explained/>. [Zugriff am 22 10 2023].

- [27] ArduPilot Dev Team, „Communicating with Raspberry Pi via MAVLink,“ ArduPilot, 2023. [Online]. Available: <https://ardupilot.org/dev/docs/raspberry-pi-via-mavlink.html>. [Zugriff am 22 10 2023].
- [28] Airlclip, „Welche Sensoren nutzt eine Drohne zur Stabilisierung der Fluglage?,“ [Online]. Available: <https://www.airclip.de/faq/Welche-Sensoren-nutzt-eine-Drohne-zur-Stabilisierung-der-Fluglage>. [Zugriff am 22 10 2023].
- [29] UAV Navigation, „UAV Navigation In Depth: What Is An IMU And What Is It Used For?,“ 10 2 2022. [Online]. Available: <https://www.uavnavigation.com/company/blog/what-is-an-imu>. [Zugriff am 22 10 2023].
- [30] PX4, „Holybro Kakute F7 (Discontinued),“ , 3 8 2023. [Online]. Available: https://docs.px4.io/main/en/flight_controller/kakutef7.html. [Zugriff am 22 10 2023].
- [31] L. Nagel, „What is an Electronic Speed Controller & How Does an ESC Work,“ TYTo Robotics, 26 1 2022. [Online]. Available: <https://www.tytorobotics.com/blogs/articles/what-is-an-esc-how-does-an-esc-work>. [Zugriff am 22 10 2023].
- [32] R. A. S, „What Is Raspberry Pi? Here's The Best Guide To Get Started,“ simplilearn, 12 12 2022. [Online]. Available: <https://www.simplilearn.com/tutorials/programming-tutorial/what-is-raspberry-pi>. [Zugriff am 22 10 2023].
- [33] GISGeography, „How GPS Receivers Work – Trilateration vs Triangulation,“ 31 3 2022. [Online]. Available: <https://gisgeography.com/trilateration-triangulation-gps/>. [Zugriff am 22 10 2023].
- [34] MagicMaps, „Wie funktioniert Satellitennavigation?,“ MTS Schrode AG, 2018. [Online]. Available: <https://www.magicmaps.de/gnss-wissen/wie-funktioniert-gps/?L=0>. [Zugriff am 20 10 2023].

- [35] B. Schneider, „A Guide to Understanding LiPo Batteries,“ Roger's Hobby Center, 9 10 2021. [Online]. Available: <https://www.rogershobbycenter.com/lipoguide>. [Zugriff am 22 10 2023].
- [36] Digitec, „Purecrea MT3608 Step-up Spannungsregler DC-DC Boost 2-24V,“ [Online]. Available: <https://www.digitec.ch/de/s1/product/purecrea-mt3608-step-up-spannungsregler-dc-dc-boost-2-24v-entwicklungsboard-kit-25480056?supplier=8244233>. [Zugriff am 22 10 2023].
- [37] R. Ramirez, „How To Power Your Raspberry Pi With A Battery,“ Circuit Basics, [Online]. Available: <https://www.circuitbasics.com/how-to-power-your-raspberry-pi-with-a-lithium-battery/>. [Zugriff am 22 10 2023].
- [38] Digitec, „Purecrea TP4056 Lithium LiPo Akku Batterie Lademodul Micro USB 5V 1A,“ [Online]. Available: <https://www.digitec.ch/de/s1/product/purecrea-tp4056-lithium-lipo-akku-batterie-lademodul-micro-usb-5v-1a-entwicklungsboard-kit-32964668?supplier=8244233>. [Zugriff am 22 10 2023].
- [39] MikroKopter.de, „LiPo-Grundlagen,“ [Online]. Available: <https://wiki.mikrokopter.de/LiPo-Grundlagen>. [Zugriff am 22 10 2023].
- [40] M. 2520, „Floodfill,“ Wikipedia, 14 11 2021. [Online]. Available: <https://de.wikipedia.org/wiki/Floodfill>. [Zugriff am 22 10 2023].
- [41] Sarger, „A*-Algorithmus,“ Wikipedia, 1 2 2022. [Online]. Available: https://de.wikipedia.org/wiki/A*-Algorithmus. [Zugriff am 22 10 2023].
- [42] R. Atuhaire, „Popular Shortest Path Algorithms,“ 23 6 2022. [Online]. Available: <https://blog.octachart.com/popular-shortest-path-algorithms>. [Zugriff am 23 10 2023].
- [43] P. Jaiswal, „Let's be A* — Learn and Code a Path Planning algorithm to fly a Drone,“ Medium, 30 4 2019. [Online]. Available: <https://towardsdatascience.com/lets-be-a-learn-and-code-a-path-planning-algorithm-to-fly-a-drone-4d5b566fa1ae>. [Zugriff am 22 10 2023].

- [44] A. Patel, „Heuristics,“ Stanford, [Online]. Available: <https://theory.stanford.edu/~amitp/GameProgramming/Heuristics.html>. [Zugriff am 22 10 2023].
- [45] BMU Media GmbH, „Die O-Notation,“ 2023. [Online]. Available: <https://bmu-verlag.de/die-o-notation/>. [Zugriff am 22 10 2023].
- [46] M. A. D. H. M. A. B. F. T. U. R. A. S. Adil M. J. Sadik, „A Comprehensive and Comparative Study of Maze-Solving Techniques by Implementing Graph Theory,“ IEEE Xplore, 11 2010. [Online]. Available: https://www.researchgate.net/publication/224202469_A_Comprehensive_and_Comparative_Study_of_Maze-Solving_Techniques_by_Implementing_Graph_Theory. [Zugriff am 23 10 2023].
- [47] 3D Robotics, „DroneKit Python, Taking Off,“ 2016. [Online]. Available: https://dronekit-python.readthedocs.io/en/latest/guide/taking_off.html. [Zugriff am 19 10 2023].
- [48] Autodesk, „TinkerCad,“ 2023. [Online]. Available: <https://www.tinkercad.com/>. [Zugriff am 22 10 2023].
- [49] Prusa3D, „Original Prusa i3 MK3S+ 3D-Drucker,“ Prusa Research, 2023. [Online]. Available: <https://www.prusa3d.com/de/produkt/original-prusa-i3-mk3s-3d-drucker/>. [Zugriff am 22 10 2023].
- [50] Filamentworld, „PLA,“ 2023. [Online]. Available: <https://filamentworld.de/3d-druck-wissen/was-ist-pla/>. [Zugriff am 22 10 2023].
- [51] Plastikalternative, „Was ist eigentlich PLA?,“ 2023. [Online]. Available: <https://www.plastikalternative.de/was-ist-pla/>. [Zugriff am 22 10 2023].
- [52] Global Sat, „PRODUCT USER MANUAL GPS RECEIVER ENGINE BOARD EM-406A,“ Global Sat, [Online]. Available: https://shop.boxtec.ch/pub/sparkfun/EM-406A_User_Manual.pdf. [Zugriff am 23 10 2023].

- [53] Distrelec Schweiz AG, „RASPBERRY PI 3 MODEL B+ BULK - Raspberry Pi 3 - Model B+ 1GB RAM, Raspberry Pi,“ 2023. [Online]. Available: <https://www.distrelec.ch/en/raspberry-pi-model-1gb-ram-raspberry-pi-raspberry-pi-model-bulk/p/30109158>. [Zugriff am 22 10 2023].
- [54] Spark Fun Electronic, „GPS Receiver - EM-506 (48 Channel),“ [Online]. Available: <https://www.sparkfun.com/products/12751>. [Zugriff am 22 10 2023].
- [55] A. Negi, „Raspberry Pi 3 B+ Pinout with GPIO functions, Schematic and Specs in detail,“ eTechnophiles, 26 12 2023. [Online]. Available: <https://www.etechnophiles.com/raspberry-pi-3-b-pinout-with-gpio-functions-schematic-and-specs-in-detail/>. [Zugriff am 22 10 2023].
- [56] NewBeeDrone, „Hobbywing Xrotor 4in1 ESC - 60A BLHeli_32,“ 2023. [Online]. Available: https://newbeedrone.com/products/hobbywing-xrotor-4in1-esc-60a-blheli_32. [Zugriff am 23 10 2023].

Abbildungsverzeichnis

Abbildung 1: Softwarekomponenten und deren Interaktion.	6
Abbildung 2: Gazebo open-source 3D Robotik Simulator.	6
Abbildung 3: Nicht durchdringbares Hindernis der Grösse 1m x 6m x 10m.	7
Abbildung 4: ROS stellt eine Verbindung zwischen verschiedenen Anwendungsprogrammen her.	9
Abbildung 5: LaserScan Message Type [12]	10
Abbildung 6: Visuelle Darstellung der Kommunikation zwischen Gazebo und ROS	10
Abbildung 7: ArduPilot stellt die Firmware für den FC bereit.	12
Abbildung 8: SITL ist eine ArduPilot Version, die auf dem PC läuft.	12
Abbildung 9: DroneKit ist eine Softwarebibliothek zur Kommunikation mit Drohnen	13
Abbildung 10: Koordinatensystem im Skript und Simulator.	13
Abbildung 11: Ultrasonic Distance Sensor V2.0 [24]	14
Abbildung 12: Schematische Darstellung der Funktionsweise des Ultraschallsensors. [25] ...	14
Abbildung 13: Holybro Kakute F7 Flight-Controller [30]	15
Abbildung 14: Hobbywing Xrotor 4in1 ESC [56]	16
Abbildung 15: Raspberry Pi 3 Model B+ 2017 [53]	16
Abbildung 16: In der Drohne verbautes GPS-Modul [54]	17
Abbildung 17: Purecrea MT3608 Step-Up Spannungsregler [36]	17
Abbildung 18: TP4056 Charge Controller [38]	18
Abbildung 19: Map mit Weg W von S nach Z	18
Abbildung 20: Map mit Hindernis, Startpunkt S und Zielpunkt Z.	20
Abbildung 21: Stack zu Beginn des Algorithmus.	21
Abbildung 22: Richtungen der drei möglichen Nachbarzellen	21
Abbildung 23: Drei elemente mit der Distanz liegen auf dem Stack	21
Abbildung 24: Aus dx und dy resultierende Richtungen.	22
Abbildung 25: Map mit eingetragenen Distanzen zum Ziel für jede Zelle.	23
Abbildung 26: Mögliche Richtungen in welche sich die Drohne bewegen kann.	24
Abbildung 27: Wand als Hindernis im Simulator.	25
Abbildung 28: Arme an der Basisplatte befestigt	26

Abbildung 29: Konstruktion der Arme in TinkerCad [48]	26
Abbildung 30: Motoren an das ESC angeschlossen, FC darüber angebracht.	27
Abbildung 31: Gelb: Signal; Rot: Spannung; Schwarz: Ground; Volett: Receiver (RX) und Transmitter (TX) [55].....	27
Abbildung 34: Step-Up Spannungsregler und Charge Controller.....	28
Abbildung 32: Gekreuzte RX TX Verbindung	28
Abbildung 33: Überblick über den Aufbau der Elektronik	28
Abbildung 35: Vollständig zusammengebaute Drohne	29
Abbildung 36: Testflug im Simulator mit U-förmigem Hindernis	29

Glossar

Auf die Übersetzung der englischen Begriffe wird im Text zur besseren Verständlichkeit verzichtet. Im Folgenden werden die wichtigsten Begriffe definiert.

Simulation: Die Nachahmung bestimmter realer Prozesse oder Systeme mithilfe von Modellen.

Simulator: Ein Programm, welches eine Simulation ausführt.

Hardware: Physische Komponenten eines Computersystems.

Software: Nicht physische Komponenten eines Computersystems, z.B. Daten und Programme.

Physik-Engine: Eine Softwarekomponente, welche die Simulation von physikalischen Interaktionen ermöglicht.

Python: Eine Programmiersprache.

Python Skript: Eine Ansammlung von Befehlen in der Programmiersprache Python, welche der Reihe nach ausgeführt werden

Node: Eine ausführbare Datei innerhalb von ROS, welche eine spezifische Anwendung erfüllt

Topic: Ein benannter Kommunikationskanal, über welchen Nodes miteinander kommunizieren.

Publisher: Ein Programm, welches Nachrichten sendet.

Subscriber: Ein Programm, welches Nachrichten empfängt.

Plugin: Eine Softwarekomponente, welche ein bestehendes Programm erweitert.

Personal Computer (PC): Ein Computer, welcher für den individuellen Gebrauch durch eine Einzelperson vorgesehen ist.

Application Programming Interface (API): Eine Programmierschnittstelle die zur Anbindung anderer Programme an ein Programm dient.

Quadrocopter: Ein Hubschraubertyp mit vier Rotoren.

Drohne: Synonym für Quadrocopter.

Stack: Eine Sammlung von Elementen. Entweder kann das oberste Element entfernt oder ein Element hinzugefügt werden.

Instanz: Ein konkretes Exemplar oder Objekt der Klasse

Klasse: Eine Vorlage für Objekte/Instanzen, welche die Attribute bestimmt.

Konstruktor: Initialisiert den Anfangszustand eines Objekts/Instanz einer Klasse.

Funktion: Eine Sammlung an Befehlen, die zusammen ausgeführt werden.

Argument in einer Funktion: Eine Referenz, welche beim Aufruf einer Funktion angegeben wird.

Callback-Funktion: Funktion die als Argument an eine andere Funktion übergeben wird.

Laufzeitverhalten: Gibt an, wie sich die Laufzeit eines Algorithmus oder einer Funktion in Abhängigkeit zur Grösse der Eingabe verhält.

General Purpose Input Output (GPIO): Anschluss Pins an welche man weitere Hardwarekomponenten anschliessen kann.

Polylactid Acid (PLA): Sind Polymilchsäuren, welche als Grundlage zur Herstellung eines Kunststoffes dienen. Dieser ist einer der am weitesten Verbreiteten Kunststoffe im 3D-Druck.

Ground: Ist ein Leiter, welcher ein System erdet.

Eigenständigkeitserklärung

Hiermit bestätige ich, dass ich meine Maturitätsarbeit selbstständig und nur unter Zuhilfenahme der in den Verzeichnissen oder in den Anmerkungen genannten Quellen und KI-/LLM-Tools angefertigt habe. Die Mitwirkung von anderen Personen hat sich auf Beratung und Korrekturlesen beschränkt. Alle verwendeten Unterlagen und Gewährspersonen sind vollständig aufgeführt.

Ort, Datum:

Unterschrift:

Dank

Ich möchte mich ganz herzlich bei folgenden Personen bedanken, für ihre Unterstützung und Hilfe bei jeglichen Problemen, die während der Arbeit aufgetaucht sind.

Florian Köhl, Betreuer und Lehrperson für Informatik an der Kantonsschule Uster.

Jörg Bischof, Diplom Informatiker Univ. und Engineering Manager, SRE bei Google.

Anhang

A. Komponentenliste

Komponenten für die Drohne:

- 4x Motor (bereits vorhanden)
- 1x ESC (bereits vorhanden)
- 1x FC (bereits vorhanden)
- 1x Raspberry Pi 3 B+ (bereits vorhanden)
- 1x microSD-Karte bereits vorhanden)
- 1x Frame (wird entworfen und anschliessend 3D-gedruckt)
- 1x 6s 1400 mAh Lipo-Akku für Drohne (bereits vorhanden)
- 6x Arduino Grove Ultrasonic Distance Sensor (1x vorhanden) bei [Reichelt.com](https://reichelt.com) bestellen
- 1x 1s 2400 mAh Lipo-Akku für Raspberry Pi, bei [Swaytronic.ch](https://swaytronic.ch) bestellen
- 1x Step-Up Spannungsregler für Raspberry Pi, bei [Digitec.ch](https://digitec.ch) bestellen
- 1x Charge Controller, um Lipo Akku zu schützen, bei [Digitec.ch](https://digitec.ch) bestellen

B. Setup Anleitung

Anleitung Setup Gazebo Garden und ROS2

Mein GitHub Repository : <https://github.com/Jack2153/DroneObstacleAvoidance>

1. Gazebo Garden Simulator installieren:

<https://gazebo-sim.org/docs/garden/getstarted> (passende Version für jeweilige Ubuntu Version auswählen)

2. ROS2 Humble installieren:

<https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debians.html>

3. ROS2 Workspace aufsetzen: Terminal: `sudo apt update` `sudo apt install python3-colcon-common-extensions`

`"source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash"` zum `.bashrc` file hinzufügen

Terminal: `mkdir ros2_ws && cd ros2_ws`

Terminal: `mkdir src`

Terminal: `colcon build`

"source /home/"Name"/ros2_ws/install/setup.bash" zum .bashrc file hinzufügen

4. Ein ROS2 package generieren

my_drone_test Package aus GitHub herunterladen (my_drone_test befindet sich in /ros2_ws/src/my_drone_test/my_drone_test)

Code Editor in ros2_ws/src öffnen

Terminal: cd ~/ros2_ws

Terminal: colcon build

Bei Fehler mit Setuptools folgendes machen:

Terminal: cd

Terminal: sudo apt install python3-pip

Terminal: pip3 install setuptools==53.2.0

Terminal: cd ros2_ws

Terminal: colcon build (sollte nun ohne Fehlermeldung funktionieren)

5. ArduPilot Plugin für Gazebo installieren:

<https://ardupilot.org/dev/docs/sitl-with-gazebo.html>

6. ArduPilot installieren: (in gleichen Workspace wie ardupilot_gazebo)

siehe Link oder untenstehende Befehle: <https://ardupilot.org/dev/docs/building-setuplinux.html#building-setup-linux>

Terminal: git clone --recurse-submodules https://github.com/ArduPilot/ardupilot.git

Terminal: cd ardupilot

Terminal: `Tools/environment_install/install-prereqs-ubuntu.sh -y`

Terminal: `. ~/.profile`

7. `gz_ws/src/ardupilot_gazebo/worlds` und `gz_ws/src/ardupilot_gazebo/models` mit den jeweiligen worlds und models Ordern aus meinem GitHub ersetzen:

<https://github.com/Jack2153/DroneObstacleAvoidance>

8. ROS2 und Gazebo Garden kompatibel machen:

https://gazebo-sim.org/docs/garden/ros_installation#using-a-specific-and-unsupported-gazebo-version-with-ros-2

Schritte bei Gazebo Garden befolgen

Kann sein, dass man für den zweiten Schritt noch `rosdep` installieren muss:

`sudo apt install python-rosdep2`

Beim compilen von `ros_gz` (https://github.com/gazebo-sim/ros_gz/tree/humble#from-source)

anstatt `"git clone https://github.com/gazebo-sim/ros\_gz.git -b ros2"` `"git clone https://github.com/gazebo-sim/ros\_gz.git -b humble"` verwenden.

Bei mir gibt es beim compilen immer einige Fehler mit 3 Packages, es funktioniert aber trotzdem.

9. Testen, ob alles funktioniert:

Terminal A: `startupSimSITL.py`

(<https://github.com/Jack2153/DroneObstacleAvoidance/blob/main/startupSimSITL.py>)

laufen lassen. Damit der Simulator und das ArduPilot Plugin für Gazebo gestartet werden.

Terminal B: `ros2 run ros_gz_bridge parameter_bridge`

`/lidar@sensor_msgs/msg/LaserScan@gz.msgs.LaserScan`

Terminal C: `ros2 run my_drone_test lidar_subscriber` (Wenn eine Reihe an Daten ausgegeben werden, funktioniert alles.)

C. Source-Code

Der Source-Code zum Simulator befindet sich vollumfänglich im Anhang. Den abgeänderten Source-Code für die reale Drohne findet man unter:

<https://github.com/Jack2153/DroneObstacleAvoidance>

```
#!/usr/bin/env python3
import random
import rclpy
import argparse
import math
import time
import numpy as np
from rclpy.node import Node
from sensor_msgs.msg import LaserScan
from dronekit import connect, VehicleMode, LocationGlobalRelative,
LocationGlobal
from pymavlink import mavutil

parser = argparse.ArgumentParser()
parser.add_argument('--connect', default='127.0.0.1:14550') # Specify IP-
address for the simulator
args = parser.parse_args()

# Connect to the vehicle
print('Connecting to vehicle on: %s' % args.connect)
vehicle = connect(args.connect, baud=921600, wait_ready=False)

# Function to arm and then takeoff to a user specified altitude
def arm_and_takeoff(aTargetAltitude):
    print("Basic pre-arm checks")
    # Don't let the user try to arm until autopilot is ready
    while not vehicle.is_armable:
        print ("Waiting for vehicle to initialise...")
        time.sleep(1)

    print("Arming motors")
    # Copter should arm in GUIDED mode
    vehicle.mode = VehicleMode("GUIDED")
    vehicle.armed = True
```

```

while not vehicle.armed:
    print(" Waiting for arming...")
    time.sleep(1)

print("Taking off!")
vehicle.simple_takeoff(aTargetAltitude) # Take off to target altitude

# Check that vehicle has reached takeoff altitude
while True:
    print(" Altitude: ", vehicle.location.global_relative_frame.alt)
    # Break and return from function just below target altitude.
    if vehicle.location.global_relative_frame.alt >= aTargetAltitude *
0.95:
        print("Reached target altitude")
        break
    time.sleep(1)

# Function to get the new location (longitude and latitude) based on an
original location and a north and east movement given in meters
def get_location_metres(original_location, dNorth, dEast):
    """
    Returns a LocationGlobal object containing the latitude/longitude `dNorth`
and `dEast` metres from the
    specified `original_location`. The returned LocationGlobal has the same
    `alt` value
    as `original_location`.

    The function is useful when you want to move the vehicle around specifying
locations relative to
    the current vehicle position.

    The algorithm is relatively accurate over small distances (10m within 1km)
except close to the poles.

    For more information see:
    http://gis.stackexchange.com/questions/2951/algorithm-for-offsetting-a-
latitude-longitude-by-some-amount-of-meters
    """
    earth_radius = 6378137.0 # Radius of "spherical" earth

    # Coordinate offsets in radians
    dLat = dNorth/earth_radius
    dLon = dEast/(earth_radius*math.cos(math.pi*original_location.lat/180))

    #New position in decimal degrees
    newlat = original_location.lat + (dLat * 180/math.pi)
    newlon = original_location.lon + (dLon * 180/math.pi)

```

```

# Give back the same object as the input was
if type(original_location) is LocationGlobal:
    targetlocation=LocationGlobal(newlat, newlon, original_location.alt)
elif type(original_location) is LocationGlobalRelative:
    targetlocation=LocationGlobalRelative(newlat, newlon,
original_location.alt)
else:
    raise Exception("Invalid Location object passed")

# Return target location, the new longitude and latitude for the target
Location
return targetlocation, newlon, newlat

# Function to get the distance in meters between two locations
def get_distance_metres(aLocation1, aLocation2):
    """
    Returns the ground distance in metres between two LocationGlobal objects.

    This method is an approximation, and will not be accurate over large
distances and close to the
earth's poles. It comes from the ArduPilot test code:
https://github.com/diydrones/ardupilot/blob/master/Tools/autotest/common.p
y
    """
    dlat = aLocation2.lat - aLocation1.lat
    dlong = aLocation2.lon - aLocation1.lon
    return math.sqrt((dlat*dlat) + (dlong*dlong)) * 1.113195e5

# Function to move the vehicle north a certain number of meters north and east
def goto(dNorth, dEast, gotoFunction=vehicle.simple_goto):
    """
    Moves the vehicle to a position dNorth metres North and dEast metres East
of the current position.

    The method takes a function pointer argument with a single
`dronekit.lib.LocationGlobal` parameter for
the target position. This allows it to be called with different position-
setting commands.
    By default it uses the standard method:
dronekit.lib.Vehicle.simple_goto().

    The method reports the distance to target every two seconds.
    """
    currentlocation = vehicle.location.global_relative_frame
    targetLocation = get_location_metres(currentlocation, dNorth, dEast)[0]
    targetDistance = get_distance_metres(currentlocation, targetLocation)

```

```
gotoFunction(targetLocation)

#print "DEBUG: targetLocation: %s" % targetLocation
#print "DEBUG: targetLocation: %s" % targetDistance

time.sleep(20)

# Print the location for debugging purposes
def printLocation():
    print ("Global Location (relative altitude): %s" %
vehicle.location.global_relative_frame)
    print ("Altitude relative to home_location: %s" %
vehicle.location.global_relative_frame.alt)

class FloodFillNode(Node):

    # Drone step to count the movements in the movement map
    robot_step_count = 1
    def __init__(self):
        super().__init__('flood_fill_node')

        # Create a subscriber that listens to the /sensor_right topic of ROS2
        where the right sensor publishes the data
        self.subscription_right = self.create_subscription(
            LaserScan,
            '/sensor_right',
            self.sensor_right_callback,
            10)

        # Create a subscriber that listens to the /sensor_left topic of ROS2
        where the left sensor publishes the data
        self.subscription_left = self.create_subscription(
            LaserScan,
            '/sensor_left',
            self.sensor_left_callback,
            10)

        # Create a subscriber that listens to the /sensor_back topic of ROS2
        where the back sensor publishes the data
        self.subscription_back = self.create_subscription(
            LaserScan,
            '/sensor_back',
            self.sensor_back_callback,
            10)
```

```
# Create a subscriber that listens to the /sensor_front topic of ROS2
where the front sensor publishes the data
self.subscription_front = self.create_subscription(
    LaserScan,
    '/sensor_front',
    self.sensor_front_callback,
    10)

# Initialize all the maps
self.map_distance = np.zeros((9, 9), dtype=int)
self.map_movement = np.zeros((9, 9), dtype=int)
self.map_obstacles = np.zeros((9, 9), dtype=int)

# Position the robot in the middle of the map
self.robot_x = 4
self.robot_y = 4

# Set the target altitude to 4 meters
aTargetAltitude = float(4)

# Only execute arm_and_takeoff if the current altitude of the drone is
below 90% of the target altitude
if vehicle.location.global_relative_frame.alt <= aTargetAltitude *
0.9:
    arm_and_takeoff(aTargetAltitude)

# Save the x and y coordinates of the target
self.target_x, self.target_y = self.set_target()

# Set the target via use input
def set_target(self):
    target_x = int(input("Bitte geben Sie die X-Koordinate des Zielpunkts
ein: "))
    target_y = int(input("Bitte geben Sie die Y-Koordinate des Zielpunkts
ein: "))
    return target_x, target_y

# Callback function that is executed every time the front sensor measures
a distance
def sensor_front_callback(self, msg):
    distance_to_obstacle = msg.ranges[0] # Take the first measurement as
the lidar sensor behaves the same way as a proximity sensor that only gives
back one distance value for one measurement
```

```
        if distance_to_obstacle != float('inf'): # Check for infinite values,
when the value is not infinite there is an obstacle
            distance_to_obstacle = int(round(distance_to_obstacle)) # Convert
the distance to obstacle into an integer if it is not 'inf'
            self.add_obstacle(distance_to_obstacle, sensor="front") # Add an
obstacle to the map with the right distance
            self.main_program()

    # Callback function that is executed every time the right sensor measures
a distance
    def sensor_right_callback(self, msg):
        distance_to_obstacle = msg.ranges[0] # Take the first measurement as
the lidar sensor behaves the same way as a proximity sensor that only gives
back one distance value for one measurement

        if distance_to_obstacle != float('inf'): # Check for infinite values,
when the value is not infinite there is an obstacle
            distance_to_obstacle = int(round(distance_to_obstacle)) # Convert
the distance to obstacle into an integer if it is not 'inf'
            self.add_obstacle(distance_to_obstacle, sensor="right") # Add an
obstacle to the map with the right distance

    # Callback function that is executed every time the lidar sensor measures
a distance
    def sensor_left_callback(self, msg):
        distance_to_obstacle = msg.ranges[0] # Take the first measurement as
the lidar sensor behaves the same way as a proximity sensor that only gives
back one distance value for one measurement

        if distance_to_obstacle != float('inf'): # Check for infinite values,
when the value is not infinite there is an obstacle
            distance_to_obstacle = int(round(distance_to_obstacle)) # Convert
the distance to obstacle into an integer if it is not 'inf'
            self.add_obstacle(distance_to_obstacle, sensor="left") # Add an
obstacle to the map with the right distance

    # Callback function that is executed every time the back sensor measures a
distance
    def sensor_back_callback(self, msg):
        distance_to_obstacle = msg.ranges[0] # Take the first measurement as
the lidar sensor behaves the same way as a proximity sensor that only gives
back one distance value for one measurement

        if distance_to_obstacle != float('inf'): # Check for infinite values,
when the value is not infinite there is an obstacle
            distance_to_obstacle = int(round(distance_to_obstacle + 0.1)) #
Convert the distance to obstacle into an integer if it is not 'inf'
```

```
        self.add_obstacle(distance_to_obstacle, sensor="back") # Add an
        obstacle to the map with the right distance

    # Function for the main program
    def main_program(self):
        temp_map_distance = np.zeros((9, 9), dtype=int) # Initialize a new
        temporary distance map to recalculate the distance of each cell in every step
        # Copy the obstacle array into the temporary distance array
        temp_map_distance[self.map_obstacles == -1] = -1

        # Perform the FloodFill algorithm
        self.flood_fill(self.target_x, self.target_y, temp_map_distance)

        # Refresh the main distance map
        self.map_distance = temp_map_distance
        self.map_distance[self.target_x][self.target_y] = 0 # Set the target
        to 0 in the main distance map

        # Print all the maps except the temporary map
        print(f"Distance map: \n {self.map_distance}")
        print(f"Movement map: \n {self.map_movement}")
        print(f"Obstacle map: \n {self.map_obstacles}")

        # Check if the drone is already at the target
        if self.robot_x == self.target_x and self.robot_y == self.target_y:
            print("Der Roboter ist am Ziel angekommen.")
            rclpy.shutdown() # Shutdown the ROS 2 client library
            vehicle.close() # Close the vehicle object of DroneKit

        # Move the drone in the map and in real life/simulator
        self.move_robot()
        time.sleep(2)

    # Function to add an obstacle in front of the drone at a certain distance
    in meters from the drone based on which sensor measures a distance
    def add_obstacle(self, distance, sensor):
        # If the front sensor measures a distance, there is an obstacle in the
        x-axis in front of the drone
        if sensor == "front":
            obstacle_x = self.robot_x + distance
            obstacle_y = self.robot_y
        # If the right sensor measures a distance, there is an obstacle in the
        y-axis
        elif sensor == "right":
            obstacle_x = self.robot_x
            obstacle_y = self.robot_y - distance
```

```
# If the left sensor measures a distance, there is an obstacle in the
y-axis
elif sensor == "left":
    obstacle_x = self.robot_x
    obstacle_y = self.robot_y + distance
# If the back sensor measures a distance, there is an obstacle in the
x-axis in the back of the drone
elif sensor == "back":
    obstacle_x = self.robot_x - distance
    obstacle_y = self.robot_y

# Checking if the obstacle is in the map or not
if 0 <= obstacle_x < 9 and 0 <= obstacle_y < 9:
    self.map_obstacles[obstacle_x][obstacle_y] = -1 # Setting the
obstacle to -1 in the obstacle map

# Function to perform the stack based FloodFill algorithm in a given array
with a given target
def flood_fill(self, target_x, target_y, array_to_fill):
    n, m = array_to_fill.shape # Get the dimensions of the 2D array in
which the FloodFill should be performed in

    # Define arrays to specify 4-way (non-diagonal) movement directions
    dx = [0, 0, 1, -1] # Movement directions of the x-axis
    dy = [1, -1, 0, 0] # Movement directions of the y-axis

    # Initialize stack with the target coordinates and distance 0
    stack = [(target_x, target_y, 0)]

    # Continue as long there are elements in the stack
    while stack:
        x, y, dist = stack.pop() # Remove the next cell and its distance
from the stack

        # Check if the cell should be updated
        if array_to_fill[x][y] == 0 or array_to_fill[x][y] > dist:
            array_to_fill[x][y] = dist

        # Loop to explore in all 4 directions
        for i in range(4):
            new_x, new_y = x + dx[i], y + dy[i] # Calculate the new
coordinates based on the movement direction
            # Check that the new cell is inside the array and that it
is not an obstacle (indicated by -1)
            if 0 <= new_x < n and 0 <= new_y < m and
array_to_fill[new_x][new_y] != -1:
```

```
        stack.append((new_x, new_y, dist + 1)) # Add the new
cell to the stack with incremented distance

# Function to move the drone
def move_robot(self):
    n, m = self.map_distance.shape # Get the dimensions of the 2D
map_distance array
    dx = [0, 0, 1, -1] # Movement directions of the x-axis
    dy = [1, -1, 0, 0] # Movement directions of the y-axis

    min_distance = float('inf') # Initialize the minimum distance
    min_positions = [] # Initialize a list to store positions with the
minimum distance value

    # Loop to find the neighbour cell with the minimum distance
    for i in range(4):
        new_x, new_y = self.robot_x + dx[i], self.robot_y + dy[i] #
Calculate new potential positions based on the movement direction
        if 0 <= new_x < n and 0 <= new_y < m and
self.map_distance[new_x][new_y] >= 0: # Check if new position is within the
map and that it is not an obstacle
            if self.map_distance[new_x][new_y] < min_distance: # Check if
a new minimum distance is found
                min_distance = self.map_distance[new_x][new_y] # Update
the minimum distance
                min_positions = [(new_x, new_y)] # Update the position
list with the new position
            elif self.map_distance[new_x][new_y] == min_distance: # Check
if the distance is the same as the current distance
                min_positions.append((new_x, new_y)) # Add the cell to the
position list

    if min_positions: # Check if there are any cells to move to
        new_x, new_y = random.choice(min_positions) # Randomly choose one
of the cells in the list min_positions

    # Print the current and the new location for debugging purposes
    print(f"self.robot_x: {self.robot_x}")
    print(f"self.robot_y: {self.robot_y}")
    print(f"new_x: {new_x}")
    print(f"new_y: {new_y}")

    # Calculate the movement in x and y directions
    movement_x = new_x - self.robot_x
    movement_y = new_y - self.robot_y
```

```
        self.robot_x, self.robot_y = new_x, new_y # Update the robots
current position
        self.robot_step_count += 1 # Increment the step count to display
how many steps the drone already moved
        self.map_movement[new_x][new_y] = self.robot_step_count # Set the
new position of the drone on the movement map with the value of the drone step
to indicate how many steps the drone already moved

        # Code to actually move the drone based on the calculated x and y
movement (movement_x, movement_y)
        if movement_x == 0 and movement_y == -1:
            goto(-1, 0) # Move the drone 1m to the west
        elif movement_x == 0 and movement_y == 1:
            goto(1, 0) # Move the drone 1m to the east
        elif movement_x == -1 and movement_y == 0:
            goto(0, -1) # Move the drone 1m to the north
        elif movement_x == 1 and movement_y == 0:
            goto(0, 1) # Move the drone 1m to the south
        else:
            raise Exception("Invalid Location") # Raise an exception if the
new location is invalid

# Main function
def main(args=None):
    rclpy.init(args=args) # Initialize the ROS 2 client library
    flood_fill_node = FloodFillNode() # Create an instance of the
FloodFillNode class

    rclpy.spin(flood_fill_node) # Spin the ROS 2 node to keep it alive and
responsive to callbacks

    flood_fill_node.destroy_node() # Destroy the node to end the program
    rclpy.shutdown() # Shutdown the ROS 2 client library
    vehicle.close() # Close the vehicle object of dronekit

# Check if this script is executed as a standalone script
if __name__ == '__main__':
    main() # Call the main function
```